TENANTPRO — SMART TENANT AND RENT MANAGEMENT SYSTEM



A project report submitted in partial fulfilment of the requirements of
Master of Computer Applications Post Graduate Course of
JAIN (Deemed-to-be University)

Submitted by

Anush Kulal M 24MCAR0133

Under the guidance of

Dr. Kamalraj R

Department of Computer Science and IT
JAIN (Deemed-to-be University)
Bengaluru

Date -

CERTIFICATE BY THE UNIVERSITY

Certified that the project report titled “TenantPro — Smart Tenant and Rent Management System” is based on an original project study conducted by Anush Kulal M (Register No. 24MCAR0133) of the Master of Computer Applications programme, under the guidance of Dr. Kamalraj R. The required guidance sessions have been attended. The project report has not formed a basis for the award of any Degree or Diploma of any university or any other institution.

PLACE: Bengaluru

DATE:

Dr. SAGAR GULATI
DIRECTOR
SCHOOL OF COMPUTER SCIENCE AND IT
JAIN (Deemed-to-be University)

CERTIFICATE BY THE GUIDE

Certified that the project report titled “TenantPro — Smart Tenant and Rent Management System” is based on an original project study conducted by Anush Kulal M (Register No. 24MCAR0133) under my guidance. This project has not formed a basis for the award of any Degree or Diploma of any University or any other Institution.

PLACE: Bengaluru

DATE:

Signature of the Guide

Dr. Kamalraj R

Department of Computer Science and IT

DECLARATION BY THE STUDENT

This is to declare that the project titled “TenantPro — Smart Tenant and Rent Management System” is carried out independently by me under the guidance of Dr. Kamalraj R, Department of Computer Science and IT, JAIN (Deemed-to-be University), Bengaluru. This work is an original one and has not been submitted earlier to any University or any other Institution for the fulfilment of the requirement of a course of study or any other credential.

PLACE: Bengaluru

DATE:

NAME: Anush Kulal M

USN: 24MCAR0133

ACKNOWLEDGEMENT

I am deeply indebted to Dr. CHENRAJ ROYCHAND, President, JAIN (Deemed-to-be University) Trust, Bengaluru, for having admitted me to undergo the Master of Computer Applications Post Graduate course during the academic term 2025 - 2026.

I am pleased to thank Dr. Sagar Gulati, Director, School of Computer Science and IT, JAIN (Deemed-to-be University), Bengaluru, for the facilities and the academic environment extended to me during the course of this work.

I take this opportunity to express my gratitude to Dr. Ananta Charan Ojha, Deputy Director, Dr. K Suneetha, Head of the Department of Computer Science and IT, and Dr. Murugan R, Programme Head of the MCA Programme, for their valuable guidance and support for the successful completion of the project work.

I express my profound thanks to my project coordinator Dr. I Brem Navas, Professor, Department of Computer Science and IT, and above all to my guide Dr. Kamalraj R, for guidance at the decisive points of this work and for being available whenever it mattered.

I am very thankful to my family and friends for their constant and unconditional encouragement and support.

NAME: Anush Kulal M
USN: 24MCAR0133

TABLE OF CONTENTS

Chapter	Title	Page No.
	Abstract	1
1	Introduction	3
	1.1 Overview	3
	1.2 Problem Background	3
	1.3 Motivation	4
	1.4 Objectives of the Project	4
	1.5 Scope of the Project	5
	1.6 Organization of the Report	6
2	Literature Study	8
	2.1 Introduction	8
	2.2 Review of Traditional Manual Systems	8
	2.3 Review of Existing Digital Rental Applications	8
	2.4 Payment Models on Two-Sided Platforms	9
	2.5 Review of Token-Based Session Management	10
	2.7 The Regulatory and Legal Setting	11
	2.8 Accessibility	14
	2.9 Over-the-Air Update Delivery	15
	2.10 Gap Analysis	16
3	System Analysis	17
	3.1 Existing System	17
	3.2 Limitations of the Existing System	17
	3.3 Proposed System	18
	3.4 Advantages of the Proposed System	18
	3.5 Feasibility Study	19
4	System Design and Architecture	21
	4.1 System Architecture	21
	4.2 Use Case Diagram	21
	4.3 Activity Diagrams	22
	4.4 Data Flow Diagrams	26
	4.5 Client Architecture and the View-Model	27
	4.6 API and Authorisation Layer	28
	4.7 Payment and Settlement Design	28
	4.8 Session and Authentication Design	29
	4.9 Database Design	30
	4.10 The Alignment Rule	38

Chapter	Title	Page No.
	4.11 Update Delivery	40
	4.12 Derived Alerts	41
5	System Requirements	42
	5.1 Hardware Requirements	42
	5.2 Software Requirements	42
	5.3 Programming Platform Features	43
	5.4 Functional Requirements	43
	5.5 Non-Functional Requirements	44
	5.6 Functional Requirements in Detail	44
	5.7 Non-Functional Requirements in Detail	46
6	Implementation	48
	6.1 Authentication and Session Module	48
	6.2 Property, Room and Tenant Management Module	48
	6.3 Rent Cycle and Due Date Module	49
	6.4 Payment and Confirmation Module	49
	6.5 Payment Gateway Connection Module	49
	6.6 Notification and Reminder Module	50
	6.7 Maintenance Request Module	50
	6.8 Tenant Behaviour Score Module	50
	6.9 Tenant Portal Module	51
	6.10 Room Discovery and Join Requests	51
	6.11 Account Creation by Role, and Tenants Under Eighteen	52
	6.12 First-Run Guidance	53
	6.13 The Alerts Surface	53
	6.14 Delivering Updates	54
	6.15 Testing	55
	6.16 Testing Strategy	61
7	Results and Screenshots	64
	7.1 Landlord Overview	64
	7.2 Tenant Roster	65
	7.3 Rooms and Properties	66
	7.4 Ledger	67
	7.5 Connecting a Payment Gateway	68
	7.6 Tenant Home	69
	7.7 Paying Rent	70
	7.8 Tenant Profile and Payment History	71
	7.9 Maintenance	72

Chapter	Title	Page No.
8	Conclusion and Future Enhancement	74
	8.1 Conclusion	74
	8.2 Limitations of the Present Work	75
	8.3 Future Enhancement	76
	References	79
	Appendix a — Api Endpoint Inventory	81
	A.1 How to read this appendix	81
	A.2 Landlord authentication (/api/auth)	81
	A.3 Landlord data (/api/owner)	81
	A.4 Payments (/api/payments)	82
	A.5 Properties (/api/properties)	82
	A.6 Tenant authentication (/api/tenant-auth)	83
	A.7 Tenant portal (/api/tenant-portal)	83
	A.8 Tenants (/api/tenants)	84
	A.9 Rooms (/api/units)	84
	A.10 Gateway webhooks (/api/webhooks)	84
	A.11 Declared on the application directly	85
	Appendix B — Database Schema	86
	B.1 How to read this appendix	86
	B.2 owners	86
	B.3 properties	86
	B.4 units	87
	B.5 tenants	87
	B.6 document_requests	88
	B.7 push_tokens	88
	B.8 leases	89
	B.9 expenses	89
	B.10 payment_settings	89
	B.11 rent_invoices	90
	B.12 tenant_users	90
	B.13 payments	90
	B.14 password_resets	91
	B.15 settlements	91
	B.16 maintenance_requests	92
	B.17 maintenance_messages	92
	B.18 join_requests	92
	B.19 tenant_documents	93

Chapter	Title	Page No.
	B.20 demo_state	93
	B.21 contact_verifications	93
	B.22 sessions	94
	B.23 listings	94
	Appendix C — Test Suite Inventory	96
	C.1 How to read this appendix	96

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of Reviewed Categories of Existing Systems	9
2.2	Structural Options for Moving Money on a Platform	10
3.1	Existing System Compared with the Proposed System	18
3.2	Feasibility Analysis Summary	20
4.1	Evolution of the Session Design	29
4.2	Principal Tables and Their Role	30
4.3	owners: the landlord account	32
4.4	properties: a building belonging to a landlord	32
4.5	units: a room inside a property	33
4.6	tenants: a tenancy, which is the landlord's record of a person	33
4.7	tenant_users: the tenant's own login account	34
4.8	payments: one rent payment, claimed or confirmed	34
4.9	payment_settings: one row per landlord, holding the gateway connection	35
4.10	maintenance_requests: a repair ticket shared by both parties	36
4.11	join_requests: a tenant asking to be let into a property	36
4.12	sessions: one row per issued refresh token	37
5.1	Hardware Requirements	42
5.2	Software Requirements	42
6.1	Test Suites and Assertion Counts	55
6.2	Defects Found by the End-to-End Runs	58
6.3	Defects Found During Later Verification	59
6.4	Defects Found by Verifying the Documentation Against the Code	60
A.1	Landlord authentication	81
A.2	Landlord data	81
A.3	Payments	82
A.4	Properties	82
A.5	Tenant authentication	83
A.6	Tenant portal	83
A.7	Tenants	84
A.8	Rooms	84
A.9	Gateway webhooks	84
A.10	Routes outside the routers	85
B.1	owners	86
B.2	properties	86
B.3	units	87
B.4	tenants	87
B.5	document_requests	88

Table No.	Title	Page No.
B.6	push_tokens	88
B.7	leases	89
B.8	expenses	89
B.9	payment_settings	89
B.10	rent_invoices	90
B.11	tenant_users	90
B.12	payments	90
B.13	password_resets	91
B.14	settlements	91
B.15	maintenance_requests	92
B.16	maintenance_messages	92
B.17	join_requests	92
B.18	tenant_documents	93
B.19	demo_state	93
B.20	contact_verifications	93
B.21	sessions	94
B.22	listings	94
C.1	Every suite in the build	96

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	System architecture: client, API, data, supporting services, and the money path outside all of them	21
4.2	Use case diagram: the landlord, the tenant, and the two external systems	22
4.3	Activity diagram for the system as a whole, across the landlord, system and tenant swimlanes	24
4.4	Activity diagram for paying rent: the two routes converging on a single settlement step	25
4.5	Data flow diagram, level 0: the context diagram	26
4.6	Data flow diagram, level 1: processes, data stores and the external systems	27
4.7	Entity relationship diagram of the core schema	31
7.1	Landlord overview: collection against expectation, overdue total, occupancy, six-month revenue and the open-ticket queue	65
7.2	Tenant roster with counted filters, due-state chips and behaviour scores	66
7.3	Room list showing occupancy, rent and tenant assignment per unit	67
7.4	Ledger of recorded payments with method, date and tenant	68
7.5	The online payments sheet: what it costs, what the gateway will ask for, and the key fields	69
7.6	Tenant home: rent due with countdown, deposit held, tenure, open requests and the landlord's contact	70
7.7	The payment screen with a gateway connected: the online route above the divider and the UPI route beneath it	71
7.8	Tenant profile: tenure, rent share, deposit, the behaviour score with its factors, and the payment timeline	72
7.9	Tenant help screen listing maintenance requests with their status	73

ABSTRACT

Rental housing is one of the largest parts of the urban economy in India and one of the least organised. Paying guest establishments, private hostels and small apartment blocks house a very large share of the students and working people in every city, and almost all of them are still run on paper. Names go into a register at the front desk. Rent arrives by UPI transfer or in cash. The record of a payment is a screenshot sitting on somebody's phone. A tenant who has paid holds no receipt their landlord has ever acknowledged, and a landlord with six rooms usually cannot say with confidence who is behind this month.

TenantPro was built to replace that arrangement. It has two sides: a landlord application and a tenant portal, both served from one React Native codebase running on Expo SDK 54, talking to a Node and Express API over MySQL. The system is deployed and in use. At the commit documented here it comes to 55,499 lines across 178 source files, exposes 96 API endpoints over 23 database tables, and presents 28 screens.

One decision shapes the rest of the design. Money never passes through the platform. Rent goes straight from the tenant to the landlord, either by UPI into the landlord's own address or through a payment gateway connected to the landlord's own merchant account. That keeps the operator outside the Reserve Bank of India's payment-aggregator perimeter, which carries an authorisation requirement and a net-worth floor in the tens of crores, and it means the platform bills nobody and holds nobody's funds. The catch is that an application which does not touch the money also cannot see it. So a payment a tenant reports starts life as a claim and stays one until a second party vouches for it: the landlord tapping Confirm, or the gateway answering a direct query from the server. Both of those routes end in the same settlement function, so they cannot disagree about when a month has cleared. A payment the landlord enters themselves is the exception and is recorded as confirmed outright, because it is the landlord's own statement about their own money and there is nobody left to verify it against.

The build carries 750 assertions across twenty-two test suites. Of those, 185 run end to end against a live server and a real MySQL instance, and 260 drive the compiled application inside a browser engine. Those runs found sixteen defects that unit tests over mocks had not: a cross-account authorisation hole, a landlord being asked on signup whether they were over eighteen, a screen that promised to text a guardian when nothing in the system could send a message, and two icon-only controls that a screen reader

announced as nothing but "button". All sixteen are fixed. This report sets out the architecture, the features as they actually stand, the payment and session designs, how the system was tested, and what has been identified but not built.

Keywords: Rent Management, Property Management System, Tenant Portal, Direct Payment Model, Payment Aggregator Regulation, UPI, Razorpay Payment Links, Refresh Token Rotation, React Native, Expo, Node.js, MySQL, End-to-End Testing.

CHAPTER 1 — INTRODUCTION

1.1 Overview

Rented accommodation carries a large part of the urban housing load in India. Paying guest establishments, private hostels and small apartment blocks of anywhere between two and twenty rooms are where most students and most young working people live. Nearly all of these properties belong to individuals rather than to companies, and nearly all of them are managed without software of any kind.

A typical property runs out of a register. Names are written in when somebody moves in and struck out when they leave. Rent comes by UPI or in cash, and the only trace of it is a transaction screenshot the tenant forwards over WhatsApp. Deposits are remembered rather than written down. Maintenance complaints arrive as phone calls and are either dealt with or forgotten, with nothing recorded either way. None of this is a problem at one property with four rooms. It becomes one at two properties with twelve.

TenantPro replaces that patchwork with a single record both sides can see. The landlord gets a roster of properties, rooms and tenants, with due dates worked out from each tenancy's own anniversary. The tenant sees the same records from the other direction: what they owe, when it falls due, what they have paid, and which of it the landlord has acknowledged. The client is a React Native application shipped as an Android build. The server is a Node and Express API over MySQL, deployed and running.

1.2 Problem Background

Anybody who has run a rented property for a year or two will recognise the problems with doing it by hand. Rent amounts get recorded inconsistently, or not at all. A payment made on the first and a payment made on the nineteenth look identical in the register once both have been ticked. When a tenant insists they have paid and the owner cannot find the entry, neither side has anything useful: the tenant has a screenshot proving a transfer happened, which says nothing about which month it was meant for.

Deposits are the worst version of the same thing. A deposit is collected at the start of a tenancy that might run two years and is settled at the end of it. In between, the only record is a figure in the owner's head or a line in a notebook that has since been thrown out.

Arguments at move-out are common, and the tenant usually loses them, because the tenant has no record at all.

On the owner's side, what limits growth is the lack of a consolidated view. One property can be held in your head. Three cannot, and the first thing to go is a reliable answer to the simplest question in the business: who has not paid. Reminders then go either to everybody, which irritates the people who have already paid, or to nobody, which is why collection slips.

1.3 Motivation

The idea came from watching both sides fail at the same task for opposite reasons. Owners described losing the first week of every month to chasing payments they could not confirm. Tenants described paying on time and then being asked again a week later with no way to prove the first payment had landed.

The surrounding conditions have also changed a great deal in the last five years. UPI has made digital payment ordinary rather than exceptional, so the movement of money is already solved and does not need reinventing. Cloud hosting is cheap. Practically every tenant carries a smartphone. What is missing is not a way to pay but a shared record of what the payment was for.

Looking at the applications already in this space showed a consistent split. The simpler ones are digital notebooks with no tenant-facing side, which leaves the tenant exactly where the paper register left them. The more ambitious ones fix the record problem by routing the rent through their own account and settling it on to the landlord afterwards. That solves visibility by creating a regulatory problem, and it also means the owner's rent spends time in somebody else's account. Neither approach suited the situation this project set out to serve.

1.4 Objectives of the Project

The project was shaped around a set of objectives.

1. To replace manual record keeping with a structured digital record of properties, rooms, tenants, tenancies, rent, deposits and payments, held in a relational database instead of a register.

2. To give the tenant a real view of that record, so that the person paying the rent can see what is owed, what has been paid and what the landlord has formally acknowledged.
3. To work out each tenancy's due date from its own start date rather than from a shared calendar month, so that somebody who moved in on the nineteenth is due on the nineteenth.
4. To make digital payment the ordinary route, both by UPI straight into the landlord's account and through a gateway connected to the landlord's own merchant account.
5. To keep the platform out of the flow of funds completely, so that it never holds or transmits a tenant's money and does not fall inside the regulatory perimeter that applies to payment aggregators.
6. To make sure no payment is ever treated as settled on the payer's own say-so, by requiring confirmation from the landlord or the gateway before a rent cycle moves forward.
7. To send rent reminders automatically on the channels tenants actually use, and to give maintenance complaints a shared thread so that they are recorded rather than remembered.
8. To build a session design in which a session can actually be ended, so that a stolen phone, a tenant who has moved out or a password change has some real effect.
9. To test the result by driving every endpoint against a real database and the compiled application against a real browser engine, instead of relying on unit tests over mocks.

1.5 Scope of the Project

The scope covers the operational life of an independently owned rental property. That means registering a landlord account, creating properties and rooms, bringing tenants on board either by direct entry or by an invite code they redeem, recording rent and deposit terms, generating due dates, sending reminders, accepting and confirming payments, keeping a searchable ledger, running maintenance threads, holding identity documents, computing a payment-behaviour score both parties can see, recording notice periods and leave dates, and giving the tenant a portal where all of it is visible from their side.

Two things have been added since the design was first set down, and the scope has widened to take them in. A landlord may publish a vacant room, and a tenant without a tenancy may search what has been published and ask to join one. And a tenancy now

produces a readable summary of its own recorded terms, which the tenant can open from their portal.

Some things were deliberately left out, and the boundaries are worth stating precisely, because two of them have moved.

The system does not hold or move money and is not an escrow. That has not changed and is the decision the rest of the architecture rests on.

On agreements, the system now generates a summary of the terms it already holds — the property and room, the rent, the deposit, the start date and when rent next falls due — and presents it to the tenant. It does not draft a tenancy, does not store an executed one, and does not register anything. The screen says as much in its own words: it is a summary of what has been recorded, not a stamped or registered deed. The distinction matters, and the application is the one making it rather than this report making it on the application's behalf.

On listings, the system is still not a marketplace and still does no brokerage, but the earlier statement that it has no listing side is no longer true. A landlord can publish a room that is empty, and a tenant can search what has been published and ask the landlord about it. What makes this not brokerage is what is absent: no commission, no fee for an introduction, no ranking sold to anybody, and no party to the conversation but the two who would be in it anyway. The platform publishes what a landlord asked it to publish and then gets out of the way, which is the same posture it takes with the rent.

The rest stands. There are no accounting exports and no tax filing. Expense and vendor management are outside the present scope. And the client is an Android application; no iOS build has been made.

1.6 Organization of the Report

Seven chapters and three appendices follow. Chapter 2 reviews the manual practices and the digital applications already in this space, then the regulatory, legal, accessibility and update-delivery material that constrains any design here, and ends with a gap analysis. Chapter 3 analyses the existing arrangement, lists its limitations, describes what is proposed instead, and reports a feasibility study run along four axes. Chapter 4 covers the architecture, the use case, activity and data flow models, and the design of the payment, session, database, layout, update and alert layers. Chapter 5 states the hardware, software,

functional and non-functional requirements, the last two enumerated in full. Chapter 6 goes through the implementation module by module and reports the testing, its results and the strategy behind it. Chapter 7 shows the finished interface with screenshots from the running application. Chapter 8 concludes, states the limitations honestly, and sets out what has been identified for later. Appendix A lists every endpoint the server declares, Appendix B the database schema, and Appendix C every test suite in the build; all three are generated from the source rather than maintained by hand.

CHAPTER 2 — LITERATURE STUDY

2.1 Introduction

A project like this starts by looking at what already exists. Three bodies of material mattered here: the informal practices that currently run the independent rental sector, the digital applications built to replace them, and the regulatory and security literature that constrains how anybody may build in this space. The third turned out to matter more than the second, because the central architectural decision in this project follows directly from it.

2.2 Review of Traditional Manual Systems

There is a good deal of management literature on why small businesses stay on manual systems even when digital alternatives are cheap and available. The findings repeat. What the owner weighs is not the price of the software but the time and risk of moving to it. The owner is usually also the operator and has no spare capacity for a migration. And manual systems fail quietly, so nobody ever measures what they cost.

Work on small hospitality and rental businesses makes a related point: manual records understate actual losses. A missed payment, a partial payment nobody followed up, a deposit returned in full when deductions were due — none of these show up as losses anywhere, because there is no record to compare them against. The owner's sense of the business is therefore reliably rosier than its accounts would be, which weakens the case for changing anything.

The useful conclusion for this project was not that owners are being irrational. It was that any replacement has to be usable on day one without a migration project, and has to show its value in a way the owner can see. Several decisions later in the design follow from that, in particular the choice to let a landlord run the whole system with no payment gateway connected and still get the benefit of the record.

2.3 Review of Existing Digital Rental Applications

A fair number of applications now target the Indian rental market. They range from simple mobile apps built by individual developers, through mid-tier rent-collection products, up to full property management suites aimed at organised co-living operators. Reviewing them showed a split that runs right through the category.

The simpler applications are digital registers. They give the owner somewhere to type what they would otherwise have written, which is a genuine improvement, but there is no tenant-facing side at all. The tenant ends up where they started, with no acknowledged receipt and no view of their own history. Several also model a property as a flat list of tenants, with no notion of a property holding rooms holding occupants, which makes them unusable past one small building.

The more capable products do have a tenant side, and they solve the confirmation problem convincingly, because they collect the rent themselves. The tenant pays the platform, the platform marks the payment received and then settles the money on to the landlord. From a software point of view this is clean. It also places the platform squarely inside the regulatory perimeter discussed in the next section, and it means the landlord's rent passes through an account that is not theirs on the way to them. Several of the owners we spoke to during this project objected to that without being prompted.

Table 2.1 — Comparison of Reviewed Categories of Existing Systems

Category	Tenant-facing view	Property hierarchy	Handles money	Suitability for independent owner
Paper register	None	Implicit, in the owner's head	No	Workable at one small property only
Simple digital notebook apps	None or read-only list	Flat tenant list	No	Breaks down beyond one property
Rent collection platforms	Yes	Partial	Yes, collects and settles	Regulated activity; owner loses direct settlement
Organised co-living suites	Yes	Full	Yes	Priced and scoped for operators, not individuals
TenantPro	Full parallel portal	Property to room to tenant	No, payment is direct	Designed for this segment

2.4 Payment Models on Two-Sided Platforms

Any platform sitting between a payer and a payee has three structural options, and choosing between them is a regulatory decision before it is a technical one.

Under the aggregator model the platform receives the customer's money into its own account and then pays the merchant. The Reserve Bank of India's directions on payment aggregators make this a regulated activity. An aggregator cannot keep customer funds in its ordinary account. The money has to sit in a segregated escrow with a scheduled

commercial bank, it earns the platform no interest, and it has to be settled out within prescribed timelines. Getting the authorisation at all means clearing a net-worth floor in the tens of crores. Ride-hailing platforms are the familiar example.

Under split settlement the platform does not hold the money itself. Instead it instructs a licensed gateway to split an incoming payment between a platform account and a linked sub-merchant account. The gateway holds the licence and runs the know-your-customer checks on the sub-merchants. Razorpay Route is the common implementation in India. This takes the onboarding burden off the platform and is attractive at scale, but funds still pass through a platform-controlled account, and the 2025 directions restrict how such arrangements may be put together.

Under the direct model the platform never touches the money. The payer pays the payee. All the platform does is present the right amount, the right destination and a reference, and record what happened afterwards. Namma Yatri is the best-documented Indian precedent: a ride-hailing service where the fare goes straight from rider to driver and the platform takes no commission, sustaining itself some other way. TenantPro sits here.

Table 2.2 — Structural Options for Moving Money on a Platform

Model	Who holds the funds	Authorisation required	Example
Aggregator	The platform, in a ring-fenced escrow at a scheduled commercial bank	RBI payment aggregator licence	Uber, Ola
Split settlement	The gateway, settling to linked sub-merchant accounts	Held by the gateway, not the platform	Razorpay Route
Direct	Nobody; the payer pays the payee	None	Namma Yatri, TenantPro

2.5 Review of Token-Based Session Management

The security literature on sessions for mobile clients turns on one trade-off: stateless tokens against stateful ones. A signed access token of the JSON Web Token family is checked by its signature alone, with no database lookup. That is what makes it fast and what lets it scale. The price is that nothing can take it back. A stolen device, a tenancy that has ended, a password somebody just changed — none of these affect a token already in circulation. It runs until it expires.

Current practice, set out in the OAuth 2.0 security guidance, pairs a short-lived access token with a long-lived refresh token held on the server, so that revocation has

somewhere to act. Two further properties are recommended. Rotation: presenting a refresh token spends it and returns a replacement, so a long-lived secret is never sent across the network over and over. And reuse detection: since rotation means the honest client is always holding the newest token, a token that has already been spent turning up again implies that a copy exists somewhere it should not, and the safe answer is to end every session on that account.

The literature is equally clear that the client half is where this usually goes wrong. A client that signs the user out on any authentication-looking error will sign them out on a dropped connection. A client that fires several requests at once on a cold start, all failing on the same expired token, will issue several refreshes, and all but the first will replay a spent token — which a correctly implemented server reads as theft. Both failure modes were designed against here and both are covered by tests.

2.7 The Regulatory and Legal Setting

Most of the design decisions in this project were taken under constraints that come from outside software. They are set out here because a reader who does not know them will read several later choices as arbitrary when they are not. The account below is deliberately cautious. Where a point rests on commentary rather than on the text of a regulation, it says so, and nothing here should be read as legal advice.

2.7.1 Who may hold somebody else's money

The Reserve Bank of India regulates entities that collect payments from customers and pass them on to merchants. An entity that does this is a payment aggregator, and operating as one requires authorisation, a minimum net worth running into the tens of crores of rupees, and the holding of collected funds in an escrow account with a scheduled commercial bank. The framework has been revised more than once since it was first issued, and the direction of travel has been towards bringing more arrangements inside it rather than fewer.

The practical consequence for a small rental application is blunt. The moment the platform receives a tenant's rent and later pays it to the landlord, it is doing the thing the framework describes, whatever it calls itself and however briefly it holds the money. The alternative is not to receive it at all, which is the route taken here and the reason the confirmation model exists in the form it does.

It is worth being precise about what the platform does do, because "does not touch the money" can sound like a slogan. The application computes the amount owed, names the destination — a UPI address belonging to the landlord, or a payment link issued against the landlord's own merchant account — and records what the parties say happened. The rupees move between the tenant's bank and the landlord's bank through rails the platform has no part in. No account belonging to the operator appears anywhere in that chain.

2.7.2 The gateway route, and why it is not an exception

Where a landlord connects a payment gateway, money does pass through an intermediary, but the merchant account is the landlord's own. The landlord completes that gateway's verification in their own name, the funds settle to the landlord's bank on the gateway's schedule, and the platform's role is to ask the gateway to issue a payment link and later to ask it what happened. The gateway is the regulated entity; the landlord is its merchant; the platform is neither.

The arrangement that would change this is split settlement, where a platform collects through its own account and the gateway divides the proceeds between the platform and the merchant. That is attractive because it removes the onboarding friction described in Chapter 8, and it is not pursued here, because it moves the operator into the category this design exists to stay out of.

2.7.3 Personal data

The Digital Personal Data Protection Act, 2023 governs the handling of personal data in India, with rules issued subsequently and obligations that commence in stages. The duties most relevant to a system of this kind are familiar in shape to anyone who has met comparable regimes elsewhere: collect for a stated purpose, collect no more than that purpose needs, keep it no longer than the purpose requires, and secure it.

Two design choices in this project follow directly. The first is the decision not to widen the registration form, described in Chapter 6: a field asked for before it is needed is personal data held without a purpose to justify it, and the survey of comparable platforms found none of them asking for an address, an employer or an income at account creation either. The second is the handling of identity documents, where the landlord's ability to read a tenant's document is tied to the tenancy and ends with it. A former landlord retaining a readable copy of a former tenant's identity document is exactly the retention this kind of obligation is aimed at.

2.7.4 Identity documents, and Aadhaar in particular

Indian renting practice asks for an identity document, and in most cases that document is Aadhaar. The legal position around it is more restrictive than everyday practice suggests. The governing Act restricts the sharing of core biometric information outright, restricts the publication or public display of an Aadhaar number, and limits who may perform authentication against the Aadhaar database and under what permission. Private-entity authentication in particular has a history: a provision permitting it was struck down by the Supreme Court in 2018, and a later amendment reopened a narrower route that requires application and approval rather than being available on request.

The design here avoids the whole area rather than navigating it. The application never authenticates anybody against Aadhaar, never asks for an Aadhaar number as a field, and treats Aadhaar as one of five acceptable documents rather than as a requirement — passport, driving licence, voter identity card and PAN are equally acceptable, and a tenant may satisfy the requirement with whichever they have. A document is held as an uploaded image, with the visibility rules described above, and no number is parsed out of it.

This is a conservative reading, and a more permissive one is available. It was taken deliberately, on the view that a student project has no business building on a permission it has not obtained, and that a landlord looking at a photograph of a document is doing exactly what they would do at the front desk.

2.7.5 Who may sign a tenancy

The Indian Contract Act, 1872 requires a party to a contract to be competent to contract, and a person who has not attained majority is not. The effect for rented accommodation is that a tenant under eighteen cannot hold a tenancy in their own name, and an agreement purporting to bind them is open to challenge.

This is not an academic point for the population this project serves. Students in paying guest accommodation include people in their first year of higher education who have not yet turned eighteen, and a system that signs them up as ordinary tenants is recording something that does not hold. The application asks the question at registration and, since the work described in Chapter 6, records who consents on the minor's behalf and surfaces that person to the landlord before the landlord accepts them. What it does not do is

attempt to capture or verify the consent itself, which is a document-and-signature problem rather than a form-field problem.

2.7.6 Tax, and why it shapes a field that does not exist yet

Two parts of Indian income tax law touch rented accommodation directly. A tenant paying rent above a monthly threshold is required to deduct tax at source under section 194-IB and to file the corresponding form, which requires the landlord's permanent account number. Separately, an employee claiming house rent allowance must report the landlord's permanent account number to their employer once annual rent crosses a stated figure, or furnish a declaration instead.

Neither obligation falls on the platform. Both fall on people the platform serves, and both require a piece of information the platform does not currently hold. That is the reason the landlord's permanent account number appears in Chapter 8 as identified future work rather than as a field somebody forgot: it is needed for a rent receipt to be useful to a salaried tenant, and collecting it has a purpose that can be stated, which is the test the previous section sets.

2.7.7 Verification of tenants by the police

Several Indian states and cities require a landlord to have a new tenant verified by the local police, and the requirement is usually framed as an obligation on the landlord rather than on the tenant. The published guidance varies between jurisdictions, as do the forms, the fees and the consequences of not doing it, and secondary sources disagree about how strictly the obligation is enforced in practice.

The application does not attempt this, and the disagreement in the sources is the reason. A feature that told every landlord that verification is mandatory would be wrong in some states; one that told them it is optional would be wrong in others. A per-state workflow keyed to the property's location is the shape such a feature would have to take, and it is recorded in Chapter 8 as identified rather than built.

2.8 Accessibility

Accessibility is not usually reviewed in a project of this kind, and it is reviewed here because three defects in Chapter 6 came out of it. The relevant guidance is the Web Content Accessibility Guidelines, which, although written for the web, states requirements that apply to any interface with the same force.

Two of its requirements bear directly on screens in this application. The first is that every control must have a name that assistive technology can report. A control whose entire content is an icon has no such name unless one is supplied, and a screen reader then announces it as "button" and nothing more, which tells the user that something is there and nothing about what it does. The application had two such controls, found as described in Chapter 6.

The second is that content a sticky header obscures must remain reachable, and in particular that a header must not cover the element a person is typing into. This is the requirement behind the focus clamp discussed at length in the testing chapter, and the defect found there is a textbook instance: the clamp was correct until a second bar appeared above the header, after which a field being typed into could finish underneath it.

The broader observation from this project is that these two requirements have an unusual property. A control with no accessible name is also a control an automated test cannot address by name, and must instead reach by guessing coordinates. The accessibility defect and the testability defect are the same defect. That makes a certain amount of accessibility work self-enforcing, since a suite that insists on addressing controls by name will not compile against an interface that has not named them — but only a certain amount, and Chapter 8 argues for a direct audit rather than relying on the coincidence.

2.9 Over-the-Air Update Delivery

A mobile application that ships as a store package has a release cycle measured in days, because a package has to be built, submitted, reviewed and then installed by each user. For a product with a small number of users and a short list of defects, that cycle is the dominant constraint on how quickly anything can be fixed.

The framework used here supports a second channel. Where a change affects only the JavaScript of the application and not its native code, a new bundle can be published and applied by installed copies without a new package. The cycle for such a change is minutes, and the user's involvement is a restart.

The constraint that makes this coherent is the runtime version. A published bundle is offered only to installations whose native runtime matches the one it was built against, which is what stops a bundle that depends on a newly added native module from reaching a build that does not have it. The consequence, and it is the reason Chapter 6 has a section

on the subject, is that there are two categories of update with different mechanics, different user experiences and different honest things to say about them, and an application that presents them as one thing will eventually promise an installation it cannot perform.

2.10 Gap Analysis

Putting the three strands together, the gap is reasonably clear. Manual systems still dominate the independent rental sector despite being demonstrably lossy, because the cost of switching is real and the losses are invisible. The simpler digital applications cut the owner's effort but leave the tenant where they were, which means the dispute the system most needs to prevent is the one thing it does nothing about. The more capable platforms do give the tenant a real view, but they get there by taking custody of the rent, which is regulated and is in any case unattractive to the owners this project is for.

What nobody is offering is a system that gives both sides a shared, authoritative record of a tenancy while leaving the money exactly where it already flows, from the tenant's account into the landlord's. That is the gap TenantPro fills, and the confirmation model described in Chapter 4 is how it stays accurate about payments it is deliberately unable to watch.

CHAPTER 3 — SYSTEM ANALYSIS

3.1 Existing System

The arrangement we found across the properties studied for this project is a physical register at the property, a notebook or spreadsheet with the owner, a WhatsApp thread per tenant, and the owner's memory holding the rest together. New tenants go into the register with a phone number and a room. Rent terms are agreed verbally, or in a short written agreement that is then filed and never looked at again. Payment is by UPI or cash. A UPI payment produces a confirmation in the tenant's own banking app, which they screenshot and send across; a cash payment produces nothing at all.

Reminders go out by hand, usually in a burst at the start of the month, and usually to everybody, because working out who has already paid takes longer than just sending the message. Maintenance complaints come in by phone or WhatsApp and are tracked, if at all, in the same thread as everything else. Deposits are recorded once at the beginning and recalled at the end. Notice periods are settled in conversation.

At best a diligent owner keeps all this running with the occasional error. More often entries are missed, payments get chased twice, a deposit figure is disputed at move-out, and a complaint made three weeks ago has been forgotten by both parties.

3.2 Limitations of the Existing System

Accuracy goes first. Every figure is copied by hand at least once and some are never written down at all. Timeliness goes next: the owner's picture of who has paid lags reality by however long it takes to reconcile a pile of screenshots against the register. Evidence is one-sided, since the tenant holds proof that a transfer happened but no proof of what it settled, while the owner holds a register entry the tenant has never seen.

Scalability does not degrade gradually. It fails at a threshold, somewhere around the second or third property, which is exactly where an owner would otherwise expand. There is no accountability, because nothing records who entered what or when. And the tenant, who is after all the paying customer of the business, gets no interface whatsoever. That last point is the root of most of the disputes the arrangement produces.

3.3 Proposed System

The proposed system swaps the register and the notebook for a relational record, and puts an application over it for each party. Properties hold rooms, rooms hold tenancies, and each tenancy carries its own rent, deposit, start date and split across occupants. Due dates come from the tenancy's own start date, so a tenant who moved in on the nineteenth is due on the nineteenth and not on the first.

Payments are recorded against the due date they settle, captured at the moment the payment is declared. A payment enters as a claim and becomes real only when a second party vouches for it. Where the landlord has connected a gateway, the second party is the gateway, asked directly by the server using that landlord's own credentials. Where they have not, it is the landlord, who confirms or rejects with a reason the tenant can read. Either way the same settlement function moves the tenancy on.

The tenant portal is a parallel application, not a stripped-down copy of the landlord's. It answers a different question, and it is reached through a separate account type with its own table and its own token role. Reminders go out automatically across email, push, SMS and WhatsApp. Maintenance requests become threads the two parties share, with status changes recorded in sequence alongside what was said. A payment-behaviour score is computed once and shown identically to both sides, so neither is arguing with a number the other cannot see.

3.4 Advantages of the Proposed System

Figures are computed rather than copied, so the arithmetic stops being a source of error. The landlord's view of collection is built from the same rows the tenant's view is built from, which removes the lag. Confirmation becomes an explicit recorded act instead of an assumption, so both sides hold the same evidence. A second and third property cost nothing but data entry. And every payment, confirmation, rejection and status change is a row with a timestamp on it, which is as much accountability as a system of this size needs.

Table 3.1 — Existing System Compared with the Proposed System

Aspect	Existing system	Proposed system
Record of tenancy	Register entry and memory	Relational record, visible to both parties
Due date	Shared calendar month, applied loosely	Derived from each tenancy's own start date

Aspect	Existing system	Proposed system
Proof of payment	Tenant's screenshot of a transfer	Confirmed payment row carrying the due date it settled
Who confirms	Nobody; assumed on sight of a screenshot	Landlord, or the gateway asked server-to-server
Reminders	Sent by hand, usually to everybody	Automated daily job, addressed to those actually due
Deposit record	Remembered	Stored, and shown on the tenant's own screen
Maintenance	Phone call, tracked informally	Shared thread with recorded status transitions
Tenant's view	None	Full portal: dues, history, receipts, score, documents
Scalability	Fails at two or three properties	Unaffected by portfolio size

3.5 Feasibility Study

Feasibility was examined along four axes. The fourth is not one of the standard three, but it decided more about this project than the others did.

3.5.1 Technical Feasibility

Everything the system needs already exists and is well documented. React Native with Expo gives a single codebase targeting Android, with over-the-air updates for JavaScript changes. Node and Express over MySQL is an unremarkable server stack. Payment links, transactional email, push notifications and messaging are all reachable as ordinary HTTPS APIs. Nothing here had to be built from first principles. One technical constraint did shape the design and is picked up again in Chapter 4: native modules cannot be delivered over the air, so any feature needing a native software development kit would hold back the release cadence of everything queued behind it.

3.5.2 Economic Feasibility

Running costs are small. Hosting for the API and the database, transactional email, push delivery and media storage all sit within free or low tiers at the scale of a few properties, and they scale gracefully after that. Because the platform processes no payments, it carries no payment-processing cost of its own. Where a landlord connects a gateway, the gateway's fee is charged by the gateway to that landlord against their own merchant account; the platform is not a party to it and bills nothing. The one recurring cost identified on the roadmap is a verified sending domain for transactional mail, at roughly a thousand rupees a year.

3.5.3 Operational Feasibility

This is the axis we examined most carefully, because the people who will use the system are not necessarily comfortable with software. Two design consequences came out of it. The first is that the system has to be fully useful with no gateway connected, since gateway onboarding runs through a third party's verification process and takes days; a landlord who cannot use the product until that finishes is not coming back. The second is that any screen asking for a commitment has to state its cost and its prerequisites before the user starts, because a landlord who expects five minutes and runs into a week concludes the application is broken rather than that the gateway is slow.

3.5.4 Regulatory Feasibility

The regulatory axis settled the architecture. Collecting rent centrally and paying it out is payment aggregation under the Reserve Bank of India's directions, with the escrow obligations and the net-worth floor described in Chapter 2. A student project cannot meet those conditions. Neither can the individual landlords this system serves. So it was designed so that the question never comes up: the money moves straight from the tenant to the landlord, by UPI or through the landlord's own merchant account, and the platform neither receives it nor tells anybody to disburse it.

Table 3.2 — Feasibility Analysis Summary

Axis	Finding	Consequence for the design
Technical	All components mature and documented	Native modules avoided so that updates ship over the air
Economic	Low fixed cost; no payment-processing cost borne by the platform	Platform bills nobody; gateway fees are the landlord's own
Operational	Users are not necessarily technical; onboarding friction is fatal	System fully usable with no gateway connected
Regulatory	Central collection would be payment aggregation	Direct tenant-to-landlord payment; platform outside the flow of funds

CHAPTER 4 — SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

TenantPro uses a three-tier design. The presentation tier is a React Native client that serves the landlord application and the tenant portal from one codebase. The application tier is a Node and Express API organised as routers, each with its role guard mounted once at the top. The data tier is MySQL, with 22 tables covering accounts, properties, rooms, tenancies, payments, maintenance threads, documents, notifications and sessions.

Around the three tiers sit the supporting services: a transactional mail provider, push delivery, media storage for identity documents and maintenance photographs, and the payment gateway. The last one is drawn differently in the figure below, for a reason. Every piece of information about a tenancy goes through the server. No part of the payment does. The dashed band in the figure is the money path, and it touches the tenant and the landlord and nothing in between.

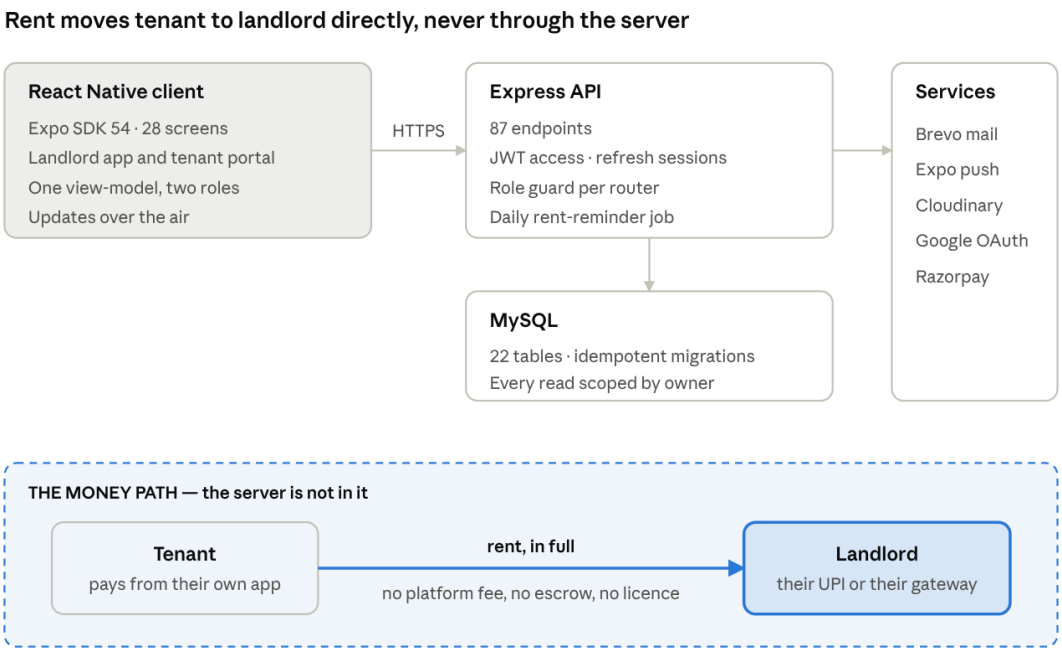


Fig 4.1 — System architecture: client, API, data, supporting services, and the money path outside all of them

4.2 Use Case Diagram

The use case diagram identifies the two primary actors and the two external systems the application talks to. The landlord and the tenant both sign in and hold a session, and from

there their paths separate almost completely: the landlord describes and administers the portfolio, while the tenant looks at their own position in it and acts on it.

Two things in the diagram are worth pointing out. The payment gateway appears as an external actor rather than as part of the system, which is the modelling consequence of the decision described in Chapter 2: the system asks the gateway questions and never handles the money itself. And the messaging services are external too, because the system composes a reminder but does not deliver it.

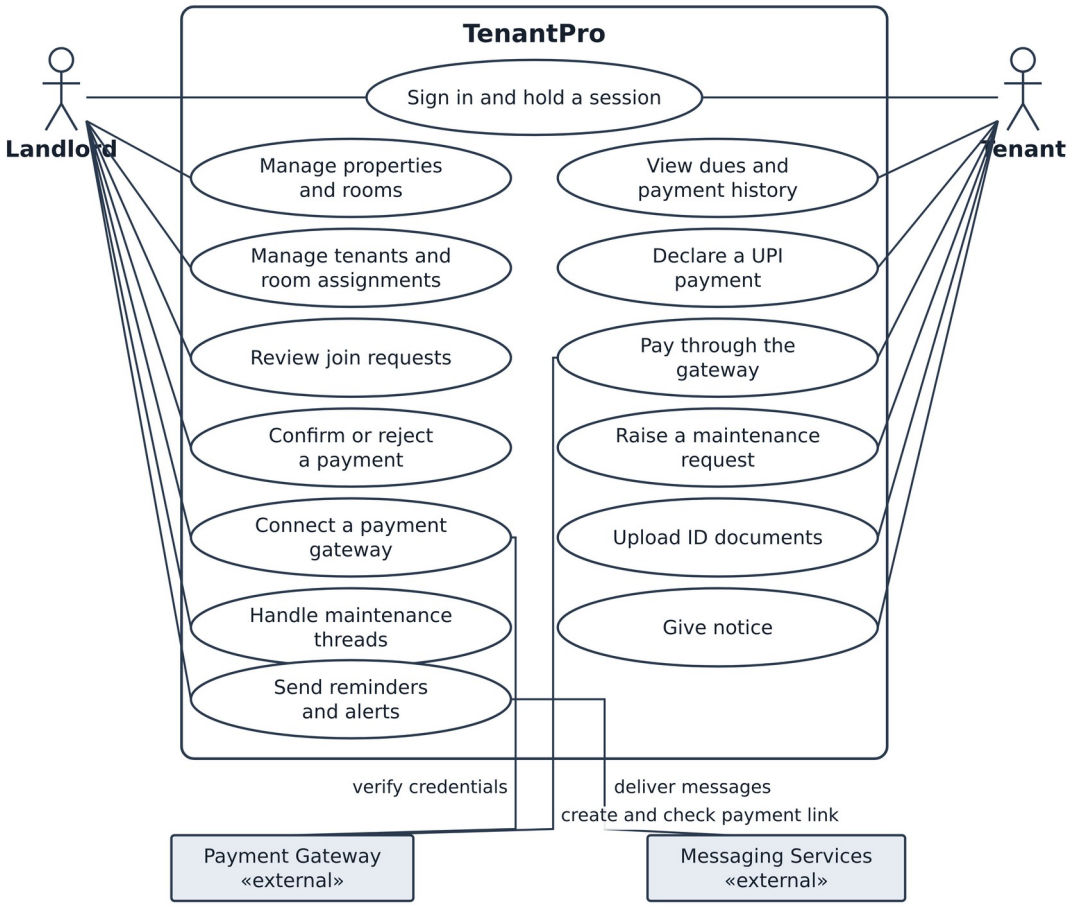


Fig 4.2 — Use case diagram: the landlord, the tenant, and the two external systems

4.3 Activity Diagrams

Two activity diagrams are given. The first takes in the system as a whole; the second zooms into the one flow that carries the most design weight.

Figure 4.3 follows a tenancy from end to end across three swimlanes: what the landlord does, what the system does on its own, and what the tenant does. Reading down the lanes shows how little of the work sits with any one party. The landlord registers, builds the

portfolio and decides on a join request. The tenant joins, looks at what is owed, pays and raises tickets. In between, the system issues the session, derives the due date from the tenancy start date, runs the morning reminder job, settles payments and computes a leave date.

Three branches in the diagram end early rather than carrying on, and that is deliberate. A join request can be declined. A payment can be rejected, or can sit as a claim nobody has vouched for yet. Both are ordinary outcomes, not errors, and a diagram that showed only the happy path would be describing a different system from the one that was built.

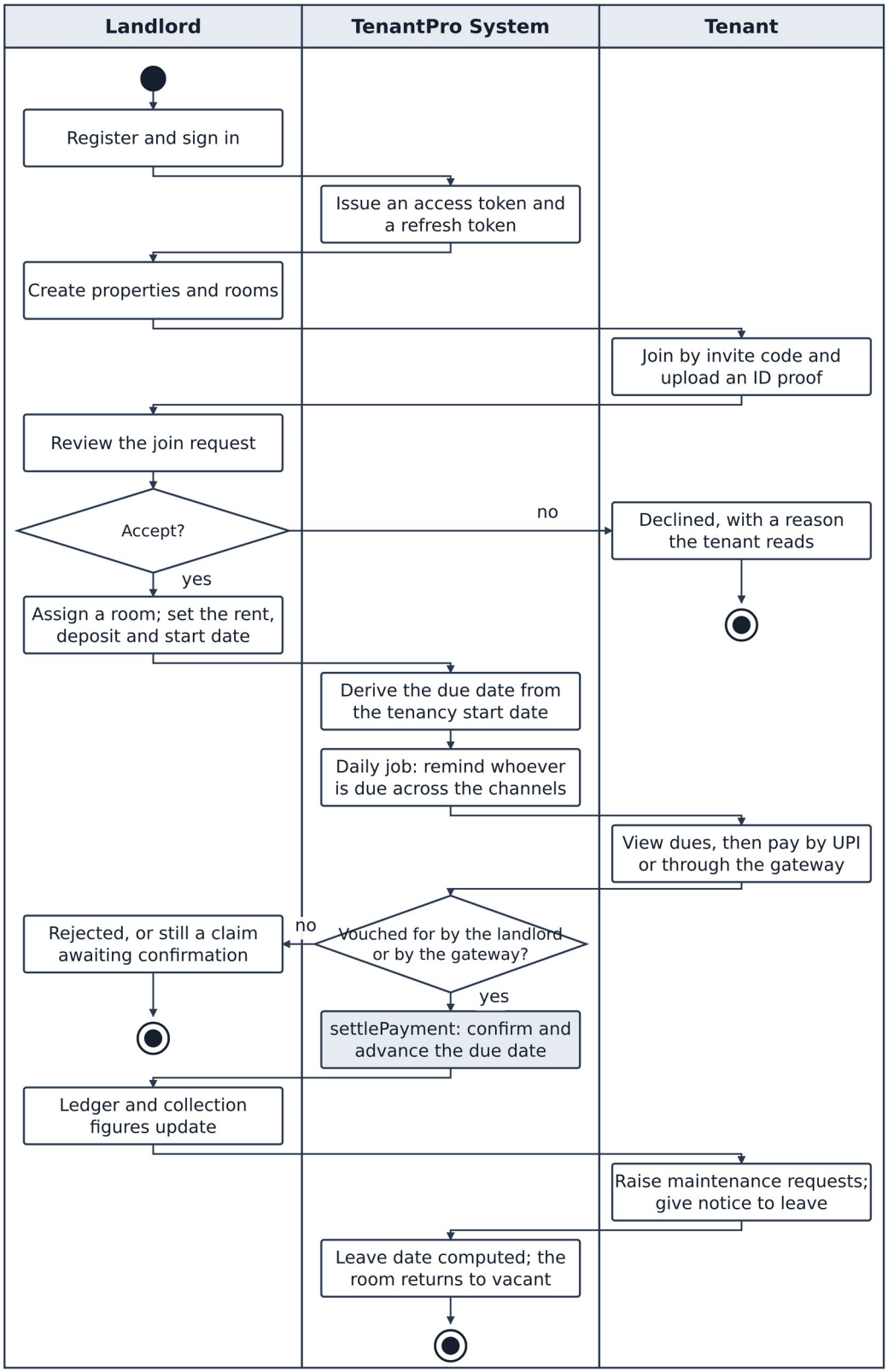


Fig 4.3 — Activity diagram for the system as a whole, across the landlord, system and tenant swimlanes

Figure 4.4 takes the payment step out of that flow and expands it. The single decision at the top, whether the landlord has connected a gateway, is what sends the flow down one of two branches.

On the UPI branch the money moves outside the system entirely, so the tenant declares the payment afterwards and it sits as a claim until the landlord confirms or rejects it. On the gateway branch the server can ask the gateway directly, so the confirmation comes from the gateway rather than from a person. The important feature of the diagram is that the two branches converge: whichever route was taken, settlement happens in exactly one place, inside one transaction that both changes the status and advances the due date.

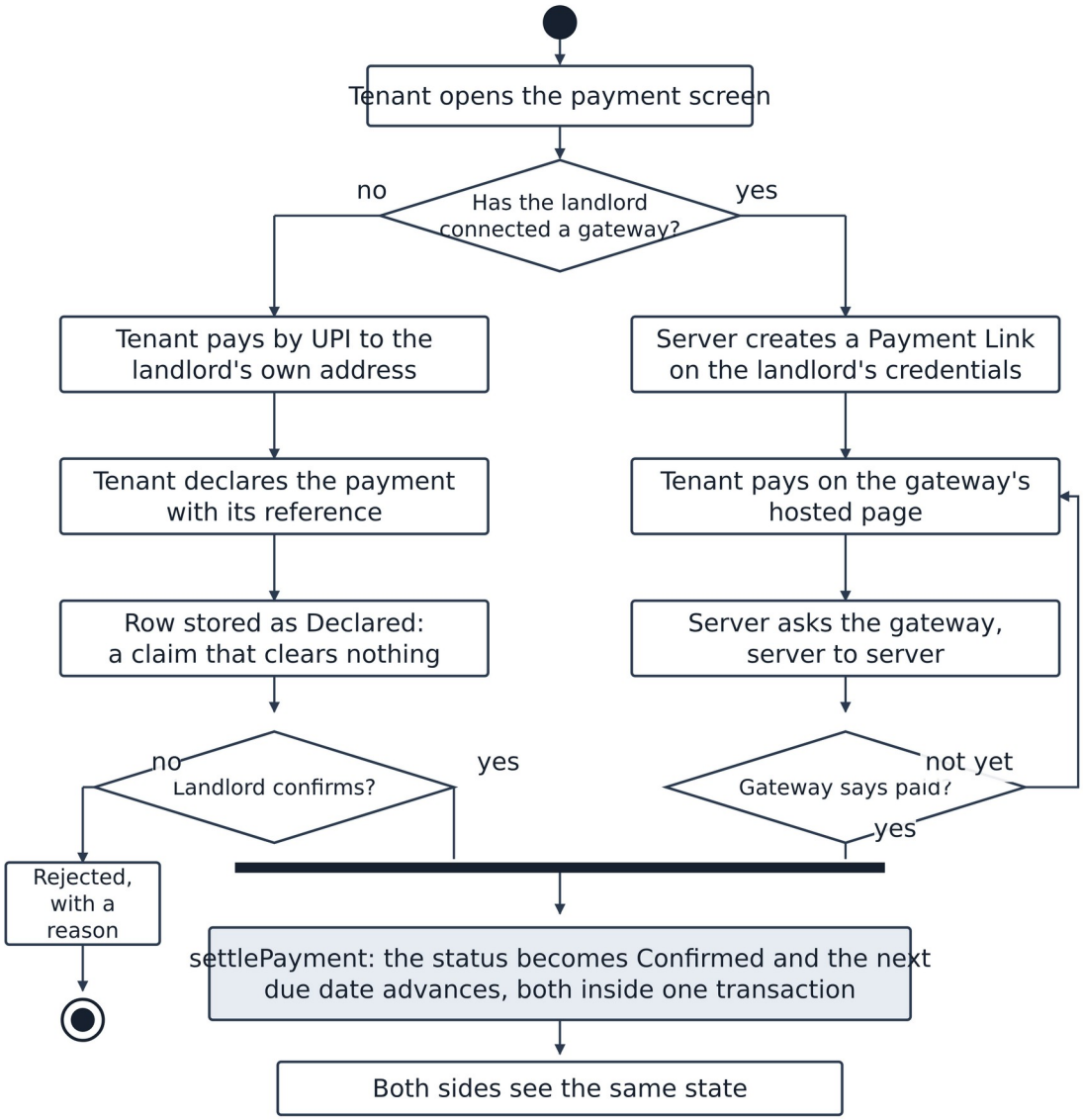
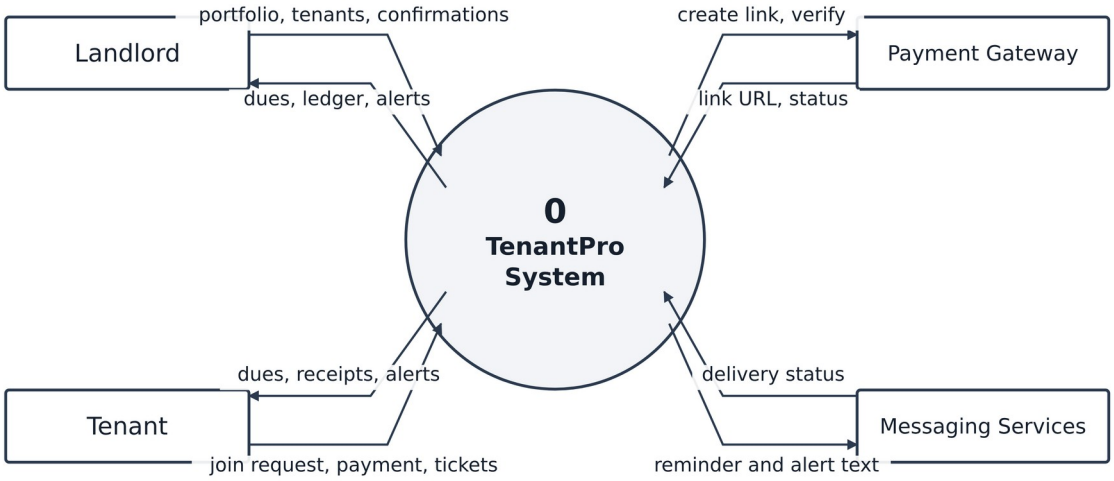


Fig 4.4 — Activity diagram for paying rent: the two routes converging on a single settlement step

4.4 Data Flow Diagrams

The context diagram fixes the boundary of the system. Four external entities surround it: the landlord and the tenant, who supply and consume information, and the payment gateway and the messaging services, which the system calls out to. The flows are all information. No flow on this diagram carries money, which is the point the architecture turns on.

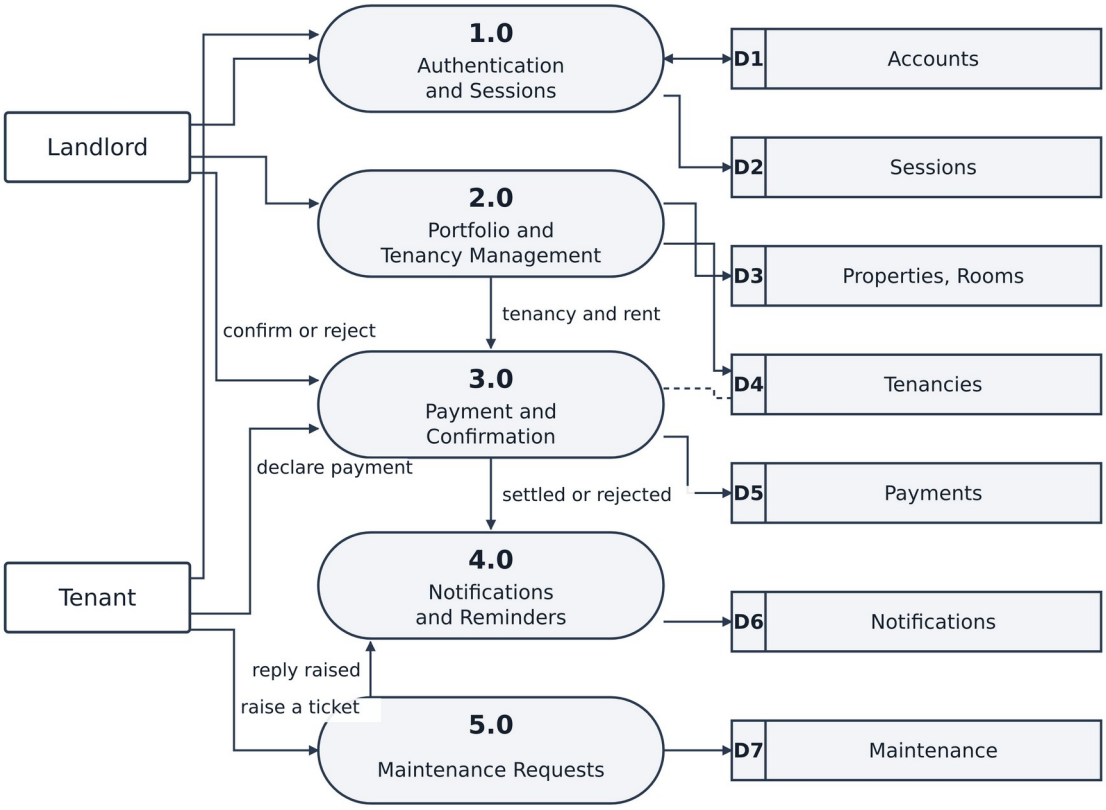


No flow on this diagram carries money: the gateway settles directly to the landlord.

Fig 4.5 — Data flow diagram, level 0: the context diagram

The level 1 diagram breaks the single process of the context diagram into five: authentication and sessions, portfolio and tenancy management, payment and confirmation, notifications and reminders, and maintenance requests. Seven data stores sit beside them.

The diagram also records a constraint that matters for correctness. Several processes read the tenancy store, but only process 3.0 writes a due date to it, and it does so only as part of the settlement transaction. Allowing any other process to move a due date would make it possible for a tenancy to advance without a payment behind it.



Dashed: process 3.0 reads tenancy rows and is the only process that advances a due date.

Fig 4.6 — Data flow diagram, level 1: processes, data stores and the external systems

4.5 Client Architecture and the View-Model

One React Native codebase serves both roles. Rather than spreading presentation logic across screens, the application runs the current state and the server payload through a single pure function that produces a view-model. A screen then holds layout and very little else.

Two things follow. Because the function is pure and needs no device, the behaviour of the interface can be asserted directly, and one of the suites in Chapter 6 does exactly that. And two screens showing the same fact cannot drift apart, since both read the same derived value instead of each working it out for itself.

The delivery mechanism imposed a constraint that shaped what got built. Updates reach an installed client through the Expo updates mechanism, which can swap out JavaScript but not native modules. Anything depending on a native software development kit can only go out in a full store release, and would hold back every other change waiting behind

it. Features of that kind were turned down wherever a design existed that ships like everything else. The payment integration is the clearest case and is covered in section 4.7.

4.6 API and Authorisation Layer

The API is organised as routers, each with its role guard mounted once at the top instead of repeated in every handler. Landlord and tenant accounts live in separate tables with their own auto-increments, so knowing who somebody is does not by itself settle what they may do. A tenant token on a landlord route is turned away by the guard, and tenant handlers scope every query through the link from the tenant account to the tenancy record.

Ownership checks are a second and separate layer. Every read is scoped by owner, but a write that accepts an identifier from the request body needs an explicit check that the row it names belongs to the caller. Those checks now live in one helper module rather than being written out at each site. There is a reason for that, recorded in Chapter 6: when they were written out individually, one of three endpoints was missed.

Schema changes apply themselves at boot, and apply safely more than once. The schema file runs its table creation statements conditionally on every start, and column-level migrations run behind existence guards. No deploy has needed a manual migration step.

4.7 Payment and Settlement Design

There are two payment routes and both are direct.

The first is UPI. The tenant's screen carries the landlord's own virtual payment address, the exact amount and a generated reference, encoded as a scannable code and as an intent that hands off to whichever UPI application the tenant already uses. The money goes from the tenant's bank to the landlord's. Nobody pays a fee, including the platform.

The second is a payment gateway. A landlord can connect their own Razorpay merchant account by entering their key and secret, which are checked against the gateway before being stored and are then sealed with authenticated encryption and never returned to any client. When a tenant pays this way the server creates a payment link against that landlord's own credentials; the tenant pays on the gateway's hosted page and the money settles to that landlord. The gateway charges its fee to the landlord through the landlord's own account. The platform bills nothing, because it is party to nothing.

Payment links were chosen over the gateway's React Native software development kit for the reason given in section 4.5. The kit is a native module. A link is plain REST over HTTPS, so the payment feature ships over the air like every other change.

The confirmation model ties the two routes together. A payment enters as a declared row, which is a claim and clears nothing. It becomes real when a second party vouches for it: the landlord tapping Confirm, or the gateway being asked outright, server to server, with that landlord's credentials, and answering that the link was paid. Both paths end in the same settlement function, which flips the status and moves the tenancy's next due date forward inside one transaction. Write that twice and the two copies will eventually disagree about when a month has cleared, so it is written once.

The gateway gets polled rather than trusted to call back. A webhook would mean every landlord configuring a URL and a shared secret in a dashboard, and most of them would never finish doing it. The signed webhook path exists and is still supported; it is just the secondary route.

4.8 Session and Authentication Design

The session design went through three versions. The sequence is worth recording, because each of the first two fixed something real and created the next problem while doing it.

Table 4.1 — Evolution of the Session Design

Stage	Access token	Can be revoked	Failure it solved	Failure it left
Original	7 days, fixed	No	—	Weekly logout for every user
Sliding	30 days, renewed on use	No	The weekly logout	Nothing could be cancelled
Current	2 hours plus a refresh token	Yes	Revocation	Lag of up to two hours on the access path

Raising the lifetime from seven days to thirty did not stop people being logged out. It made it happen monthly. A fixed lifetime counts from the last sign-in, which says nothing about whether somebody is still using the application. Open the app every morning and you were thrown out on day thirty, exactly like somebody who had not opened it since February. So the server now re-issues the access token on any authenticated request once it is past half its life, handing it back on a response header the client picks up. The new

token's lifetime comes from the old one's own issue and expiry claims rather than from a constant, which is what lets a guest session keep its own length.

A session is now a database row as well as a signature. The row holds a refresh token stored as a SHA-256 hash rather than as the token itself, since the only thing ever done with it is a comparison. Three properties follow. Rotation: spending a refresh token gets you a replacement, so a sixty-day secret is never sent repeatedly over the network. Reuse detection: the honest client always holds the newest token, so a replay of a spent one says a copy is sitting somewhere else, and every session on that account is revoked. Revocation on password reset: changing a password now ends every other session, which is what users had assumed all along that it did.

Revocation is immediate on the refresh path. On the access path it lags by at most the access token's two-hour life. Closing that last gap would mean a database read on every authenticated request, which is a cost almost nobody pays. Two hours where it used to be thirty days is the gain.

4.9 Database Design

The schema holds 22 tables and follows the entity structure closely. Landlord accounts and tenant accounts are separate tables, which is what lets identity and authorisation be told apart at the API layer. Properties belong to landlords, rooms belong to properties, and tenancies join a tenant record to a room with their own rent, deposit, split and dates. Payments reference the tenancy and carry the due date they settled, and that last field matters more than it looks. Without it, "was this on time?" has no answer after the fact, because confirming a payment moves the next due date forward and overwrites the only evidence. Its earlier absence is why the behaviour score once reported that no tenant had ever missed a payment.

Table 4.2 — Principal Tables and Their Role

Table group	Holds	Notes
Accounts	Landlord accounts, tenant accounts	Separate tables with independent keys
Portfolio	Properties, rooms	Rooms carry rent, capacity and notification preferences
Tenancy	Tenant records, assignments, notice periods	Due dates derived from each tenancy's own start date
Payments	Declared, confirmed and rejected payments	Each row stores the due date it settled

Table group	Holds	Notes
Maintenance	Requests and thread messages	Status transitions stored as events in the thread
Documents	Identity proofs and maintenance photographs	Stored externally; previews served under short-lived tokens
Notifications	Alerts, delivery records, seen markers	Seen marker is stamped by the server, not the device
Sessions	Refresh token hashes, expiry, revocation reason	Hash only; the token itself is never stored
Gateway	Per-landlord sealed gateway credentials	Encrypted with AES-256-GCM; never returned to a client

The entity relationship diagram below shows the core of the schema with its keys and cardinalities. The chain from an owner down to a payment runs through four tables, which is why an ownership check on a write that names a room is not local to the row being written and had to be factored into a shared helper.

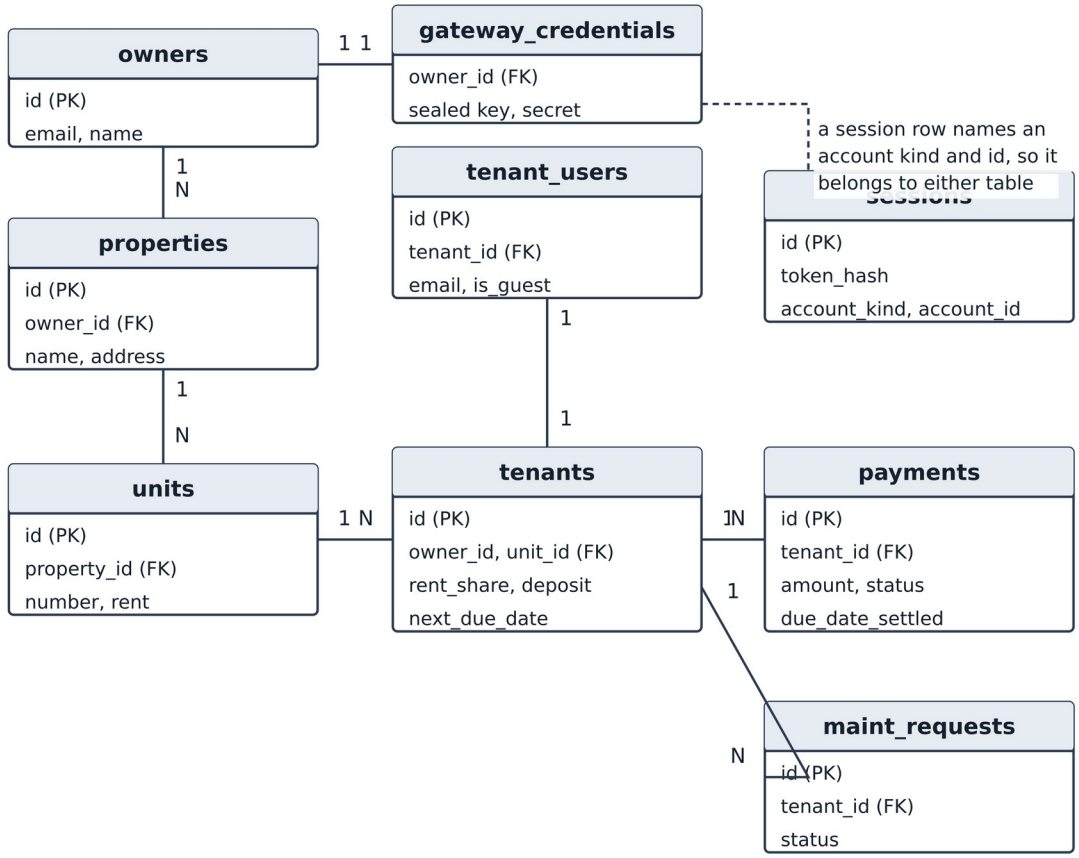


Fig 4.7 — Entity relationship diagram of the core schema

4.9.1 Table Structures

The structure of each principal table is given below, with the MySQL type, the key or constraint and what the column is for. Only the ten tables that carry the working data are

listed; the remaining twelve hold supporting records such as push tokens, password resets, document requests, contact verifications and demo state.

Table 4.3 — owners: the landlord account

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Landlord account identifier
name	varchar(100)	NOT NULL	Display name
email	varchar(100)	NOT NULL, UNIQUE	Sign-in address; also the reminder sender identity
phone	varchar(15)	NOT NULL, UNIQUE	Contact number shown to the tenant
password_hash	varchar(255)	NOT NULL	bcrypt hash; the password itself is never stored
profile_pic	varchar(255)	NULL	Avatar, held in external media storage
created_at	timestamp	DEFAULT NOW()	Registration time

Table 4.4 — properties: a building belonging to a landlord

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Property identifier
owner_id	int(11)	FK to owners, ON DELETE CASCADE	The landlord this property belongs to
name	varchar(150)	NOT NULL	Property name shown on both sides
property_type	enum	NOT NULL	PG, Apartment, Independent House or Hostel
address	text	NOT NULL	Postal address
locality, city, pincode	varchar	NULL, city defaults to Bengaluru	Location fields used for display and search
latitude, longitude	decimal(10,7)	NULL	Map pin, null until the landlord sets one; seven places is about 11 mm, and a float loses precision at the sixth
upi_id	varchar(50)	NULL	The landlord's own payment address for this property
image_url	varchar(255)	NULL	Cover photograph
created_at	timestamp	DEFAULT NOW()	Creation time

Table 4.5 — units: a room inside a property

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Room identifier
property_id	int(11)	FK to properties, ON DELETE CASCADE	The property that contains this room
unit_number	varchar(50)	NOT NULL	Room number or name
room_type	varchar(50)	DEFAULT 'Standard'	Free-text classification
capacity	int(11)	DEFAULT 1	How many occupants the room takes
rent_split_type	enum	DEFAULT 'Equal'	Equal or Custom split across occupants
base_rent	decimal(10,2)	NOT NULL	Rent for the whole room before any split
status	enum	DEFAULT 'Vacant'	Vacant, Occupied or Maintenance; follows occupancy
notify_email, notify_sms, notify_whatsapp	tinyint(1)	Defaults 1, 1, 0	Per-room reminder channels, switched independently
image_url	varchar(255)	NULL	Room photograph

Table 4.6 — tenants: a tenancy, which is the landlord's record of a person

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Tenancy identifier
owner_id	int(11)	FK to owners, ON DELETE CASCADE	The landlord who holds this tenancy
unit_id	int(11)	NULL	Assigned room; null for a tenant not yet placed
status	enum	DEFAULT 'Active'	Active or Inactive
name, phone, email	varchar	phone NOT NULL	Contact details; email is optional for guests
aadhar, company, emergency_phone	varchar	NULL	Supporting details collected at onboarding
deposit	decimal(10,2)	DEFAULT 0.00	Deposit held, shown on the tenant's own screen
rent_share	decimal(10,2)	DEFAULT 0.00	This tenant's share of the room rent
credit_score	int(11)	DEFAULT 100	Payment-behaviour score, computed from the payment history
move_in_date	date	NULL	Start of the tenancy; the anniversary the due date derives from

Field	Type	Key / constraint	Description
billing_cycle	varchar(20)	DEFAULT 'Anniversary'	How the due date is derived
next_rent_due	date	NULL	The next date rent falls due; advanced only on settlement
stay_until	date	NULL	Agreed leave date, where notice has been given
id_proof_url, image_url	varchar(255)	NULL	Identity proof and photograph
created_at	timestamp	DEFAULT NOW()	When the tenancy was created

Table 4.7 — tenant_users: the tenant's own login account

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Tenant account identifier, separate from tenants.id
name	varchar(100)	NOT NULL	Display name
email	varchar(100)	NULL, UNIQUE	Null for a guest account, which has no email
phone	varchar(15)	NOT NULL	Also the key by which an account is matched to a tenancy
password_hash	varchar(255)	NULL	Null until a guest claims a full account
tenant_id	int(11)	NULL	The tenancy this account is linked to
status	enum	DEFAULT 'Unlinked'	Unlinked, Pending or Linked
is_guest	tinyint(1)	NOT NULL DEFAULT 0	Marks an account identified by a code rather than an email
guest_code	varchar(16)	NULL, UNIQUE	The code a guest signs in with
alerts_seen_at	datetime	NULL	When the tenant last opened their alerts; stamped by the server so a wrong device clock cannot silence them
created_at	timestamp	DEFAULT NOW()	Account creation time

Table 4.8 — payments: one rent payment, claimed or confirmed

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Payment identifier

Field	Type	Key / constraint	Description
tenant_id	int(11)	FK to tenants, ON DELETE CASCADE	The tenancy this payment belongs to
amount_paid	decimal(10,2)	NOT NULL	Amount
payment_date	date	NOT NULL	Date the payment was made
due_date	date	NULL	The due date this payment settles, captured at declaration; without it punctuality is unanswerable after the fact
payment_method	varchar(50)	NOT NULL	UPI, cash, gateway and so on
reference_id	varchar(100)	NULL	Reference shown to the tenant and quoted on the transfer
status	enum	NOT NULL DEFAULT 'Confirmed'	Declared, Confirmed or Rejected
declared_by	int(11)	FK to tenant_users	Which tenant account claimed it; null when the landlord entered the row
confirmation_source	enum	NOT NULL DEFAULT 'landlord'	landlord, gateway or bank; meaningful only once status is not Declared
gateway_ref	varchar(120)	NULL, indexed	The provider's own transaction id, so a retried webhook cannot settle twice
decided_at	timestamp	NULL	When it was confirmed or rejected
decision_note	varchar(300)	NULL	The landlord's reason on a rejection, shown to the tenant
created_at	timestamp	DEFAULT NOW()	Row creation time

Table 4.9 — payment_settings: one row per landlord, holding the gateway connection

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Row identifier
owner_id	int(11)	FK to owners, UNIQUE	One settings row per landlord
upi_id, upi_number	varchar	NULL	The landlord's own payment address and number

Field	Type	Key / constraint	Description
qr_code_url	varchar(255)	NULL	A stored QR image, where one was uploaded
gateway	varchar(20)	NULL	Which provider is connected
gateway_key_id	varchar(120)	NULL	The landlord's own publishable key
gateway_secret	varchar(512)	NULL	Sealed with AES-256-GCM, useless without the key held in the environment, and never returned to a client
gateway_enabled	tinyint(1)	NOT NULL DEFAULT 0	Whether the online route is offered to tenants
updated_at	timestamp	ON UPDATE NOW()	Last change

Table 4.10 — maintenance_requests: a repair ticket shared by both parties

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Request identifier
tenant_id	int(11)	FK to tenants, ON DELETE CASCADE	Who raised it
owner_id	int(11)	FK to owners, ON DELETE CASCADE	Who has to deal with it
category	varchar(50)	NOT NULL DEFAULT 'General'	Plumbing, electrical and so on
title	varchar(150)	NOT NULL	One-line summary
description	text	NULL	The detail
priority	enum	DEFAULT 'Medium'	Low, Medium or High
status	enum	DEFAULT 'Open'	Open, In Progress, Resolved or Closed
image_url	varchar(500)	NULL	One optional photograph of the problem
created_at, updated_at	timestamp	updated_at ON UPDATE NOW()	Raised and last changed

Table 4.11 — join_requests: a tenant asking to be let into a property

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Request identifier
tenant_user_id	int(11)	FK to tenant_users	The account asking
owner_id	int(11)	FK to owners	The landlord being asked
property_id	int(11)	FK to properties	The property named by the code

Field	Type	Key / constraint	Description
unit_id	int(11)	FK to units, ON DELETE SET NULL	The room the landlord actually assigned
requested_unit_id	int(11)	FK to units, ON DELETE SET NULL	The room the applicant asked for, kept separate so accepting into a different one does not erase the request
requested_stay_until	date	NULL	How long the applicant says they are staying; null is a legitimate answer
status	enum	NOT NULL DEFAULT 'Pending'	Pending, Accepted or Rejected
source	enum	NOT NULL DEFAULT 'code'	code for a stranger typing a property code, phone_match for a number the landlord already has on file
note	varchar(300)	NULL	Message from the applicant
created_at	timestamp	DEFAULT NOW()	When it was raised
decided_at	timestamp	NULL	Null while Pending, which makes "how long has this been waiting" answerable without an event log

Table 4.12 — sessions: one row per issued refresh token

Field	Type	Key / constraint	Description
id	int(11)	PK, AUTO_INCREMENT	Session identifier
token_hash	char(64)	NOT NULL, UNIQUE	SHA-256 of the refresh token; the token itself is never stored
account_kind	enum	NOT NULL	owner or tenant, which decides the table account_id points into
account_id	int(11)	NOT NULL	The account this session belongs to
expires_at	datetime	NOT NULL, indexed	When the refresh token stops working
last_used_at	datetime	NULL	When it was last presented

Field	Type	Key / constraint	Description
replaced_at	datetime	NULL	Set when the token is spent; the spent row is kept so that reuse can be detected
revoked_at	datetime	NULL	When the session was ended
revoked_reason	varchar(20)	NULL	signout, rotated, reuse or password
device	varchar(200)	NULL	User agent, truncated and never parsed, for a future sessions screen
created_at	timestamp	DEFAULT NOW()	When the session was issued

4.10 The Alignment Rule

Twenty-eight screens laid out independently will drift. Each one chooses its own top padding, its own gutter and its own place to put a title, and the result is an application where nothing is quite wrong and nothing quite lines up. This project reached that state and then undid it, and the undoing is worth describing because it is a piece of architecture rather than a piece of styling.

The rule has four parts. The header sits directly below the system bars on every screen rather than on the landlord's screens only. Content is anchored one consistent distance below it, instead of each screen choosing a number between twelve and twenty-six. A screen's title is sticky rather than fixed: the screen keeps its full-size title on arrival, and once that has scrolled out of view a compact copy fades into the header, so the reader never loses the context and never sees it twice. And a field being typed into can never end up underneath the header.

All four live in one module. That is the point of the exercise: a rule stated in one file is a rule that can be changed in one place and tested on its own, whereas the same rule expressed as a convention in twenty-eight files is a convention that holds until somebody adds the twenty-ninth.

4.10.1 The sticky title, and why it needs a wire

The compact title in the header is drawn by the header. Whether to draw it depends on how far the body has scrolled, and the body is a sibling of the header rather than a child, so the scroll offset has to travel between two components that have no relationship in the

tree. That wire is the single piece of shared state the module introduces, and it is introduced deliberately rather than by accident: the alternative, having each screen tell the header what to show, is the drift the rule exists to stop.

The hand-off point is latched rather than computed fresh on every frame. A title that appears and disappears as the reader nudges the scroll position by a pixel is worse than one that appears slightly late, so the module uses a dead band: the switch happens a little past the threshold going down and a little before it coming back up. The value is small and the effect is that the title changes once rather than flickering.

4.10.2 The focus clamp

When the keyboard opens, the view scrolls so that the field being typed into is above it. The amount to scroll is computed from the keyboard's height and the field's position, and left alone that calculation will happily push the field up behind the header.

The clamp bounds it. It works out how much room exists between the field and the bottom of the header and permits no more scroll than that, which in the common case — a field low on a long page — changes nothing at all, and in the case that matters moves the field exactly as far as it can go and no further. A field already behind the header is not moved at all, because scrolling it would make matters worse rather than better.

The clamp has one input that was missing until late in the project: anything the shell draws above the header. The notice described in section 4.11 is such a thing, and its arrival is what exposed the omission. Anything mounted above the header now reports its height, and the clamp subtracts it. With nothing above the header the value is zero and the arithmetic is what it always was.

4.10.3 The revert flag

The rule is behind a build-time constant, as the redesign itself is and as the guided tour is. With the flag off, every screen receives the exact top padding it had before the rule existed, recovered from a table in the module rather than from anybody's memory, and the header region returns to the landlord's screens only.

Build-time rather than runtime is a deliberate choice. A runtime setting can leave a user half-migrated, with some screens following one rule and some another, and the state that produces is not one anybody designed. A constant is one value to change in one commit,

and because the module is pure JavaScript the change reaches installed copies through the update channel in minutes.

Each of the three flags is tested in both positions. A revert path that has never been taken is a claim rather than a feature, and the only way to know that turning something off restores what was there before is to turn it off and look.

4.11 Update Delivery

Chapter 6 describes what the two kinds of update look like to a user. The design question underneath is where the decision lives.

For the over-the-air kind, the application decides. It asks the update server whether a bundle exists for its own runtime, and the server answers from what has been published. Nothing in that exchange requires the application to know anything about versions, because the runtime constraint does the filtering.

For a store build the application cannot decide, because it cannot know. The update server will never mention a bundle built for a different runtime, so a phone running one version has no way to learn that a newer one exists. Something has to tell it, and the choice is between the application comparing two version strings it has been given and the server doing the comparison and sending a verdict.

The verdict is computed on the server. The reasoning is specific to what this feature is: if the comparison were wrong, every phone would either be nagged permanently or told nothing for ever, and the fix for a mistake in the application's own copy of that logic would have to arrive through precisely the channel that is not working. On the server it is a deploy. The application sends the version it is running and renders what it is told, and when it can get no answer at all it shows nothing, because silence is the right failure for a notice nobody asked for.

The comparison itself is small and is tested harder than its size suggests, for a reason the testing chapter returns to: compared as strings rather than as numbers, version 1.10.0 sorts before 1.9.0, so a release at 1.10.0 would reach nobody and nothing anywhere would look broken.

4.12 Derived Alerts

The notification list on both sides is computed rather than stored. There is no alerts table, no row written when something happens, and no background job sweeping for conditions. The list is produced from the data the application has already fetched: a payment awaiting confirmation is an alert because it is a payment awaiting confirmation, not because a row somewhere says so.

The advantages are the ordinary advantages of derivation. There is no second copy to fall out of step with the first, nothing to clean up when the underlying row changes, and no possibility of an alert that outlives the condition that produced it. A tenant whose payment is confirmed stops having an unconfirmed payment, and the alert stops existing in the same instant, because it never existed separately.

What derivation cannot produce is the knowledge of whether the user has seen any of it. Current state can say that a payment was confirmed; it cannot say that the person has not yet noticed. That single fact is the only thing in the feature that must be stored, and it is stored as one timestamp per account: the moment the list was last opened. Each row carries the moment its subject occurred, and a row more recent than the timestamp is unread.

Rows with no meaningful moment carry none and are never unread. Rent falling due on Friday is true every day of that week, and an application that counts it as news every morning is an application whose notification badge people learn to ignore. The distinction between a standing fact and an event is made in the derivation rather than in the display, which is the right place for it, because it is a statement about the data rather than about the screen.

CHAPTER 5 — SYSTEM REQUIREMENTS

5.1 Hardware Requirements

TenantPro is a client-server system. On the user's side the requirements are small: an Android phone of reasonably recent vintage and an internet connection. On the server side they are those of a small Node process and a MySQL instance, both of which run comfortably on entry-level cloud infrastructure at the scale this system is built for.

Table 5.1 — Hardware Requirements

Component	Minimum	Recommended
End-user device	Android 8.0 smartphone, 2 GB RAM	Android 11 or later, 4 GB RAM
Network	2 Mbps mobile data	4G or Wi-Fi
Application server	1 vCPU, 512 MB RAM	2 vCPU, 2 GB RAM
Database server	1 vCPU, 1 GB RAM, 10 GB storage	2 vCPU, 2 GB RAM, 25 GB storage
Development machine	4 GB RAM, 20 GB free disk	8 GB RAM or more, SSD

5.2 Software Requirements

The stack was picked to balance productivity, delivery flexibility and cost.

Table 5.2 — Software Requirements

Layer	Technology	Purpose
Client framework	React Native on Expo SDK 54	Single codebase for both roles, Android build
Client delivery	Expo updates and EAS Build	Over-the-air JavaScript updates; store builds when native changes
Server runtime	Node.js with Express	REST API, 96 endpoints across role-guarded routers
Database	MySQL	22 tables; schema applied idempotently at boot
Authentication	Signed access tokens with server-side refresh tokens	Short-lived access, revocable sessions
Encryption	AES-256-GCM, SHA-256	Sealed gateway credentials; hashed refresh tokens
Payments	Razorpay Payment Links over REST; UPI intent and QR	Direct settlement to the landlord
Messaging	Transactional email, push notifications, SMS and WhatsApp	Rent reminders and alerts

Layer	Technology	Purpose
Media	Cloud media storage	Identity documents and maintenance photographs
Testing	Plain Node scripts; Playwright with Chromium	Unit, integration, end-to-end and browser-driven suites
Version control	Git and GitHub, with GitHub Actions	Source control and build or update workflows

5.3 Programming Platform Features

React Native with Expo gives a single JavaScript codebase that runs on the device while still letting most changes go out without a store release. That second property is not a convenience. It is a constraint, and it shaped the feature set in the way described in Chapter 4.

Node with Express was picked for the server because JavaScript on both sides removes a translation layer between client and server models, and because the Express router model maps neatly onto the role-guard design. MySQL was picked over a document store because this data is strongly relational: a payment means nothing except in relation to a tenancy, which means nothing except in relation to a room and a property.

External integrations are reached through documented REST APIs with explicit error handling. One point of care is worth flagging here and comes up again in Chapter 6. A request that never reached a third party is a different event from one that reached it and was refused, and the two must not be reported to the user in the same words.

5.4 Functional Requirements

The system shall let a landlord register, sign in and recover a forgotten password. It shall let them create properties and rooms, with per-room rent, capacity, occupancy status and notification preferences. Tenants shall be creatable directly or able to join by redeeming an invite code, with the landlord accepting or declining and optionally assigning a room at the same time.

It shall work out each tenancy's due date from its own start date and maintain every tenant's due state. It shall let a tenant declare a payment, and shall record that payment as a claim carrying the due date it is meant to settle. It shall let a landlord confirm or reject a claim, with a rejection reason the tenant can read. Where a landlord has connected a

gateway, it shall create a payment link against that landlord's credentials and shall confirm the payment by asking the gateway rather than by trusting the client.

It shall keep a searchable ledger of confirmed payments, narrowable to one property. It shall send rent reminders daily on the configured channels. It shall provide maintenance threads shared between landlord and tenant, with status transitions recorded in sequence. It shall accept identity documents and serve previews under short-lived tokens. It shall compute a payment-behaviour score from the same code for both parties and itemise its factors. It shall record notice periods and compute leave dates on the server. And it shall give the tenant a portal showing their dues, history, receipts, deposit, tenure and score.

5.5 Non-Functional Requirements

Usability comes first, because the users are not necessarily technical. Screens are expected to state the cost and the prerequisites of an action before the user commits to it, and error messages are expected to tell a network failure, a refusal by a third party and a mistake by the user apart from one another.

Security requires third-party credentials to be stored encrypted and never returned to a client, refresh tokens to be stored only as hashes, sessions to be revocable, and authorisation to be enforced on the server for every write that accepts an identifier from the client. Correctness requires that a payment is never treated as settled on the payer's assertion alone, and that a status change and a due-date advance happen inside one transaction.

Reliability requires that a passing network failure never signs a user out and is never reported as a rejected credential. Maintainability requires that behaviour shared between the two applications is implemented once. Portability requires that schema changes apply themselves at boot, so that a deploy is a code deploy and nothing more.

5.6 Functional Requirements in Detail

The requirements below are stated as the system actually behaves at the commit this report documents, rather than as they were first written down. Where a requirement was specified and not built, it appears in Chapter 8 instead of here.

5.6.1 Accounts and sessions

10. A landlord and a tenant register through separate endpoints and separate forms, and one email address or mobile number belongs to one or the other and never to both.
11. Registration issues a session immediately, so a new account is signed in rather than returned to a sign-in screen.
12. A password is at least six characters, enforced on the server for both roles.
13. Each field on a registration form reports its own fault, and the summary states how many fields need attention rather than reciting the rules.
14. A tenant who states they are under eighteen must supply a guardian's name and mobile number before the account is created.
15. A session may be ended, and ending it makes the issued tokens unusable rather than merely forgetting them on the device.
16. A refresh token is single-use; presenting a spent one is treated as evidence of theft and ends the session family.

5.6.2 Properties, rooms and tenancies

17. A landlord records properties, each holding rooms, each holding occupants, so that the model matches the building rather than flattening it to a list of names.
18. A tenancy records its rent, its deposit, its start date, its billing cycle and its notice period.
19. A rent due date is computed from the tenancy's own start date, so a tenant who moved in on the nineteenth is due on the nineteenth.
20. A tenant may be brought on board by direct entry, by redeeming an invite code, or by asking to join a published room.
21. A join request shows the landlord who is asking, how to reach them, whether an identity document is on file, and, where the applicant is a minor, who consents for them.

5.6.3 Rent and payments

22. A tenant may pay by UPI into the landlord's own address or, where the landlord has connected one, through a payment gateway on the landlord's own merchant account.
23. A payment reported by a tenant is recorded as a claim and does not advance the rent cycle until confirmed.

24. A claim is confirmed by the landlord, or by the gateway answering a direct query, and both routes run the same settlement logic.
25. A payment entered by the landlord is recorded as confirmed, because it is the landlord's own statement about their own money.
26. A rejected claim carries a reason, and the tenant sees it.
27. Every confirmed payment appears in a ledger searchable by person, room, method, reference, amount or date.

5.6.4 Maintenance, documents and discovery

28. A tenant raises a maintenance request with a title and a description, and both parties see one thread per issue with every reply and status change in order.
29. A tenant holds identity documents against their account; the landlord may see that a document exists, and may read it only while the tenancy runs.
30. A landlord may publish a vacant room; a tenant without a tenancy may search what is published, filter it and ask about one.
31. A tenant may open a summary of their own tenancy terms, which states on its face that it is a summary and not a registered deed.

5.6.5 Notification and guidance

32. Rent reminders are sent automatically on a schedule, by email and by push.
33. Both portals show a derived list of what needs attention, with unread state on the tenant's side.
34. A first-time user of either portal is offered a guided tour, which is replayable from Settings afterwards.
35. The application checks for an over-the-air update on launch and on returning to the foreground, and can be asked to check on demand.
36. Where the server announces a newer store build, every screen of both portals carries a notice naming the version.

5.7 Non-Functional Requirements in Detail

5.7.1 Security

37. Gateway credentials are sealed with authenticated encryption before storage and are never returned to any client, in any form, under any role.

38. Refresh tokens are stored as hashes, so a leak of the table does not yield usable tokens.
39. Authentication and password-reset endpoints are rate limited, with the limits configurable but defaulting to the production values.
40. Every write that names a room, a property or a tenant verifies that the named object belongs to the caller, through one shared check rather than per-handler.

5.7.2 Privacy

41. No field is collected at registration beyond what registration needs.
42. A landlord's access to a tenant's identity document ends with the tenancy.
43. No Aadhaar number is requested as a field, and no authentication is performed against any national identity database.

5.7.3 Usability and accessibility

44. Every control carries a name that assistive technology can report, including controls whose entire content is an icon.
45. A field being typed into is never covered by the header.
46. Every screen states what it is; no screen relies on the user remembering which portal they are in.
47. An error message states what went wrong and what to do about it.

5.7.4 Reliability and operability

48. A malformed or incomplete request is answered with a client error and a message, never with a server error and a database string.
49. The database schema is created and migrated on boot, so a deployment requires no manual migration step.
50. A single endpoint reports the health of the database, mail, push, SMS, uploads, gateways, discovery and the announced store build.
51. Any release may be reverted: a feature by a flag and an update published over the air, or the whole bundle by republishing a previous one.

CHAPTER 6 — IMPLEMENTATION

Implementation went module by module, each module being a functional area with its own routes on the server and its own derived state on the client. They are described below roughly in the order a landlord and a tenant run into them.

6.1 Authentication and Session Module

This module registers accounts, authenticates them, issues sessions and ends them. Landlord and tenant accounts sit in separate tables, and the token issued to each carries a role the API's guards act on. Sign-in can also go through a federated Google identity.

A session is a short-lived access token plus a refresh token held as a hash in a sessions table. The access token is renewed quietly on any authenticated request once it passes half its life. The refresh token rotates every time it is used, and presenting one that has already been spent revokes every session on the account. Signing out revokes the row with a recorded reason, and a password reset revokes every other session.

The client half needed the most care. A failed request triggers at most one refresh and one replay, and only a refusal from the refresh endpoint itself ends the session; a server fault, a timeout or a dead connection does not. Parallel refreshes all share one in-flight promise, so an application firing eight requests on a cold start issues a single refresh instead of eight. Without that, seven of the eight would replay a token the first had already spent, the server would correctly read the replay as theft, and the feature built to stop lockouts would have been causing them.

6.2 Property, Room and Tenant Management Module

This is where the landlord describes the shape of the business. Properties get a name, a type and an address. Rooms get a number, rent, capacity and their own notification preferences. A room's occupancy status follows its occupants on its own.

Tenants can be created directly by the landlord or can join by redeeming an invite code, in which case the landlord sees the request with the tenant's details and any identity documents before accepting it. A tenant is allowed to exist without a room, and that state is reachable from the roster rather than tucked out of sight, because somebody added but not yet placed is an ordinary situation and not an error.

Every write in this module that takes a room identifier from the request body checks that the room belongs to the caller. That check sits in one shared helper. It did not always, and section 6.15 says what came of it.

6.3 Rent Cycle and Due Date Module

Each tenancy carries its own start date, rent and split. The due date is computed from the tenancy's own anniversary rather than from a shared calendar month, which is what lets a tenant who moved in on the nineteenth be due on the nineteenth. The module keeps every tenant's due state and exposes it as the chip shown on the roster: due in so many days, so many days late, or a leaving date where notice has been given.

Advancing the due date is not done here. It happens only inside the settlement transaction described in section 6.4, so a due date can never move without a payment having settled, and a payment can never settle without the due date moving.

6.4 Payment and Confirmation Module

A payment is created as a declared row carrying the amount, a generated reference, the method, and the due date it is meant to settle. In that state it is a claim and clears nothing.

Confirmation comes from one of two places. The landlord can confirm or reject the claim from a queue showing the context needed to decide, and a rejection carries a reason the tenant reads. Or, where the payment went through a connected gateway, the server asks the gateway directly using that landlord's credentials and settles on the gateway's own answer. Both call the same settlement function, which flips the status and moves the tenancy's next due date forward in one transaction.

On the tenant's side the distinction is made visible rather than hidden. A claim still waiting on the landlord shows in amber as awaiting confirmation, not in the green kept for money that has arrived. Show the two alike and a tenant will believe their month is clear when it is not.

6.5 Payment Gateway Connection Module

This module lets a landlord connect their own Razorpay merchant account. The sheet that does it states the gateway's fee right next to the offer, because describing one without the other is how somebody switches on a percentage fee without noticing. It also sets out what the gateway is going to ask for before the landlord starts: a permanent account

number, a bank account, one business proof, and realistically two to five working days. It adds that test keys work straight away and that tenants can carry on paying by UPI the whole time.

Credentials are checked against the gateway before anything is stored, so a mistyped secret is caught while the landlord still has the form in front of them. What gets stored is sealed with AES-256-GCM and never returned to any client. A failure to reach the gateway is reported as a network failure and not as a rejected credential; the word "rejected" is kept for the gateway actually saying no.

6.6 Notification and Reminder Module

A scheduled job runs each morning and sends rent reminders to the tenants who are actually due, across email, push, SMS and WhatsApp according to the room's preferences, with every channel phrasing the same day the same way. Beyond reminders, the module raises alerts for a landlord's reply on a maintenance thread, a payment confirmed or rejected, and the outcome of a join request.

The alert indicator works off the same payload the screen renders rather than a separate unread counter that could drift away from it. A seen marker tells an alert that is news apart from one that is merely still true, and the server stamps that marker, so a phone with the wrong clock cannot silence its own alerts.

6.7 Maintenance Request Module

A tenant raises a request with a category, a priority and a photograph if they have one, then follows it in a thread shared with the landlord. Status changes land in that thread as events, in order with whatever was said around them, so the history of a repair reads as one story instead of a conversation plus a separate status field.

6.8 Tenant Behaviour Score Module

The module computes a payment-behaviour score from the tenant's record of on-time, late and missed payments. One piece of code produces it and both sides see the same number, which is the whole point: a tenant looking at a different figure from the one their landlord sees cannot argue with either. The factors are listed out instead of being boiled down to a single dial, and where there is too little history to say anything, the screen says so. The alternative is a needle parked at zero, which reads like a verdict the tenant has earned.

6.9 Tenant Portal Module

The tenant portal is a parallel application rather than a cut-down copy of the landlord's. Its home screen carries the rent card with the amount, the property and room, the exact due date and a countdown, and underneath it the deposit held and the tenure. Both of those are things tenants routinely cannot remember and have no other record of.

The module also covers guest accounts for tenants with no email address, identified by a code with a stay window enforced on the server; turning a guest into a full account; joining a property by scanning or typing an invite code; receipts with status, reference and any rejection reason; giving and withdrawing notice, with the real dates worked out and shown before anything is written; uploading and removing identity documents; and directions to the property where the landlord has pinned it.

6.10 Room Discovery and Join Requests

A landlord with an empty room and a tenant looking for one were, until this module, two people the system held records for and could not introduce. The landlord can now publish a room. A tenant who has no tenancy sees a Find screen listing what has been published, with the rent, the locality, how many spots are left and whether the room is shared or private, and can filter and sort it.

Asking to join does not book anything. It raises a join request that lands in the landlord's queue beside the ones raised by invite code, and the landlord accepts or declines it. The tenant's side says so plainly — nothing is booked — and a room already asked about says "Request already sent" rather than offering the same button twice. That second state is small and easy to miss, and it was missed: a test asserted on the words "Ask to join" and failed the first time it ran against a database where the previous run had already asked.

The screen has a cold-start problem that no amount of code solves. A discovery page with nothing on it teaches a new user that the feature is broken rather than that the city is empty. The system therefore seeds a set of demonstration properties across Bengaluru localities, owned by accounts on a reserved domain that cannot receive mail, each holding a hash of a random string that was generated inline and never kept, so none of them can be signed into by anybody including the operator. Nothing a tenant sees marks them out, and that is a fault rather than a detail. They carry ordinary names and stock portraits, so to somebody browsing the Find screen they read as six real landlords with real rooms. The

accounts cannot be signed into and nobody can be harmed by asking to join one, but a listing that cannot be told from a real one is a listing presented as real, and it ought to carry a visible marker. The whole module sits behind a server-side flag that turns it off without a deployment, which is the blunt mitigation available today.

6.11 Account Creation by Role, and Tenants Under Eighteen

The two portals have always had separate sign-in screens and separate registration endpoints, and the application has always recorded which of them a person came through. For a long time it did nothing with that: both roles were sent to one create-account screen with one set of fields.

The consequence was visible on the landlord's form, which asked whether the person was over eighteen and told them they could sign their own agreement and pay rent directly. That is a question about whether somebody can hold a tenancy, and it has nothing to do with owning property. Each role now gets a form that says which account it is making and asks only what that role is asked. The fields themselves stayed as they were — a name, an email, a mobile number and a password — because a survey of what comparable Indian platforms ask at registration found nothing collected beyond that, and because a field asked for before it is needed is a field held without a reason to hold it.

The age question, kept on the tenant form, turned out to be doing less than it claimed. A person under eighteen is not competent to contract under the Indian Contract Act, so a minor cannot hold a tenancy in their own name; that is the entire reason the question exists. Answering "under eighteen" raised a card headed GUARDIAN REQUIRED and promised that a guardian would be sent a consent link. Nothing was collected and nothing was sent — the card was static text, the button called the same registration the adult path called, and no guardian, consent or message existed anywhere in the server. The form now asks for the guardian's name and mobile number, with the relationship optional, stores them against the account, and says what actually happens: the details go to the landlord, who must have that guardian's consent before a tenancy can start. The server refuses the registration without them, and an adult who sends guardian details anyway stores none, so "no guardian" and "a guardian we failed to capture" cannot look alike in the table.

6.12 First-Run Guidance

Somebody opening either portal for the first time sees a dock of four or five buttons, depending on which portal it is, and a screen of somebody else's records. The guided tour answers what the application is and what to do with it, in seven steps for a landlord and six for a tenant, without a manual.

It dims the screen and leaves one region lit, and it navigates as it goes, so the screen being described is the one behind the card. The lit region is measured from the component itself where that component reports its position, and falls back to a band of the shell — the header, the dock, the top of the body — where it does not. That fallback is the part worth recording: a tour that points at real components needs every component it can point at to report itself, and the first screen somebody redesigns then breaks the tour silently. Falling back to a band means a screen that stops reporting loses sharpness and nothing else, and there is no state in which the light falls on a rectangle that is not there.

The highlight travels between steps rather than cutting, and a caret on the card aims at it. That is not decoration. The eye follows a thing that moves, so a spotlight sliding from the rent card to the dock takes the reader's attention with it, where a cut leaves them hunting for what changed on a dimmed screen — which is most of the work the tour exists to save them. The tour is remembered per portal, can be left by Skip or by a close control, and can be replayed from Settings for ever after, because a tour you cannot repeat is one you will miss while distracted.

6.13 The Alerts Surface

Both portals carry a bell. What it opens is built entirely from data the application already holds — a payment waiting to be confirmed, a tenant overdue, a join request, an open repair, a room nobody is in — so the list itself needed no new endpoint and no polling of its own. One endpoint was added, and only one: the write that marks the rows seen, which is the single thing derivation cannot do.

What derivation cannot know is whether any of it is news. Current state can say "your payment was confirmed"; it cannot say "and you have not seen that yet". Each row therefore carries the moment the thing happened, and anything newer than the account's last-seen stamp counts as unread. Rows with no meaningful moment — rent falling due on Friday — carry none and never count, because a standing fact should not light a bell

every morning until Friday. The stamp is written when the sheet is opened rather than when it is closed, since somebody who opens it and swipes away has still seen it. This applies to the tenant's bell only. The landlord's side lists what needs them but has no unread state at all, an asymmetry that was not designed so much as not yet finished.

The bell is an icon with no text, which made it a control a screen reader announced as "button" and a test could only reach by guessing coordinates — the same defect wearing two faces. Its accessible name now carries its state in three forms — so many waiting on you, nothing new, or all clear — which line up with the tenant sheet's own wording in two of the three cases. An earlier version of that name called the middle state "something new", which is false — it means the list is not empty, not that anything happened — and would have told a screen-reader user there was news every time they passed the header, all week, about a date that had not moved.

6.14 Delivering Updates

Two different things wear the word "update" and only one of them can happen inside the application. A change to the JavaScript can be fetched and applied by the application itself, which takes seconds and a restart. A change to native code cannot: it needs a new package installed through the operating system's own installer, and an application that downloads and installs its own replacement is the behaviour every store rejects.

The first kind is handled by a prompt that checks on launch and whenever the application returns to the foreground, lists what is in the update from notes carried inside the update manifest, and applies it. Its Later button used to be close to a one-way door. The prompt did re-check whenever the application came back to the foreground, so backgrounding and reopening would bring it back, but there was no way to ask on purpose, and somebody who has just dismissed a thing and then wants it is unlikely to think of minimising the app to get it. Both portals now have a row in Settings that asks again on demand and answers either way, including when the answer is that there is nothing — a button that does nothing visible when there is no update is a button people decide is broken.

The second kind the application can only notice, and it cannot notice unaided: a phone running one version has no way to learn that a newer one exists, because the update server only ever offers it bundles built for its own runtime. The server is therefore asked, and the comparison is made on the server rather than in the application — deliberately,

because if that comparison were wrong, the fix would have to reach phones through exactly the mechanism that is broken. When a newer build is announced, a bar appears above the header on every screen of both portals, naming the version and the first line of what is in it, and leading to wherever it can be installed from. It is a bar rather than a sheet because there is no button here that finishes the job: the thing it announces happens in another application and probably not now, and a sheet for that would be a wall between somebody and their rent every time they opened it.

That bar produced the most instructive defect of the whole verification effort, and it is reported in the testing section below.

6.15 Testing

The build carries 750 assertions across twenty-two suites. None of them uses a test framework. Each suite is a plain Node script that exits non-zero when something fails, which keeps the test code free of dependencies and makes a failing run reproducible from one command.

Table 6.1 — Test Suites and Assertion Counts

Suite	What it drives	Isolation	Assertions
secretbox	AES-256-GCM sealing of gateway credentials	Pure	24
session	Token renewal, through the real middleware and interceptor	Stubbed database	41
gateway	Gateway controllers	Stubbed database and network	72
vm.gateway	The real view-model function, native layer stubbed	Stubbed native	52
refresh.client	The real request interceptor: refresh, retry, and when not to sign out	Stub adapter	29
layout	The alignment rule: padding, the sticky title, the focus clamp	Pure, both sides of the flag	37
ratelimit	The limiter's own limits, and that configuring them cannot disable them	Pure	13
appversion	Version comparison and the store-build verdict	Pure	37
refresh.e2e	Refresh tokens, reading the database directly	Live server and MySQL	42
api.e2e	50 of the 96 endpoints, and the authorisation boundaries	Live server and MySQL	127

Suite	What it drives	Isolation	Assertions
guardian.e2e	A tenant under eighteen, from form to column	Live server and MySQL	16
ui.e2e	Both portals end to end	Live stack and Chromium	71
ui.crossportal	A ticket reaching the landlord, with the clock running	Two browsers, live stack	42
ui.discover	Find a place: filters, sorting, asking to join	Live stack and Chromium	26
ui.alerts	The bell on both sides, and the seen stamp across a reload	Live stack and Chromium	18
ui.tour	Every control on the guided tour	Live stack and Chromium	19
ui.signup	Field-by-field validation	Live stack and Chromium	17
ui.signuproles	That the two portals do not share one form	Live stack and Chromium	14
ui.updates	Check for updates, both portals	Live stack and Chromium	15
ui.storebanner	The new-build notice, on every screen of both portals	Live stack and Chromium	20
ui.layout	Header alignment, measured rather than read	Live stack and Chromium	10
ui.signout	Signing out, and saying so while it happens	Live stack and Chromium	8
Total	—	—	750

6.15.1 Unit Testing

Unit tests cover the calculation-heavy and security-critical functions on their own: sealing and unsealing gateway credentials, the renewal arithmetic that reads a token's remaining lifetime out of its own claims, the alignment rule's padding and focus clamp, the rate limiter, version comparison, and the view-model function that turns server state into what the screens render. Refresh-token hashing is not among them; it is covered end to end by refresh.e2e, which reads the stored rows directly, rather than by a unit test.

6.15.2 Integration Testing

Integration tests run the controllers with their dependencies stubbed at the boundary rather than inside. The gateway suite is the clearest example: it runs the real controller code against a stubbed network, which is how the difference between a refused credential and an unreachable host can be asserted directly instead of inferred.

6.15.3 End-to-End Testing

The end-to-end suites run against a live server and a real MySQL instance seeded by a deterministic generator. That generator deliberately produces the rows that break screens and never turn up when you want them: a tenant with no room, a guest with no email, a guest whose stay has already ended, rent overdue and due today and upcoming and never set at all, a payment declared and undecided, another rejected with a reason, maintenance threads where the landlord spoke last and others where the tenant did, a tenant who has given notice, and a property with no rooms in it. The pseudo-random number generator is seeded, so a failing run can be reproduced exactly.

The refresh suite goes further and reads the sessions table directly, so claims about rotation, reuse detection and revocation are checked against stored rows rather than against the API's account of itself. During one live browser run a session was driven through three access-token expiries without the user being signed out. Signing out through the interface afterwards was confirmed to have written a revocation reason into the row and to have made the token unusable.

6.15.4 Browser-Driven User Interface Testing

The user interface suite runs the same JavaScript the phone runs: the same view-model, the same screens, the same request client and interceptors, laid out for the browser by the web renderer an Expo web export produces, and driven through Chromium. What it cannot reach is anything behind a native module, so the camera, the image picker, push delivery and the UPI intent hand-off are all out of its scope. For those it prints a skip rather than a pass, so a green run never claims coverage it does not have.

Running the application in a desktop browser is not a substitute for a device and is not claimed as one here. It is, though, the only automated way available in this project's environment to assert that a real screen renders a real row correctly, and it found defects no unit test had.

One figure in this chapter deserves stating rather than leaving to be inferred. The suites do not touch every endpoint. api.e2e requests about fifty distinct paths of the ninety-six the server declares, and taking the union of every suite that talks to the server still leaves roughly a third of the surface undriven — the Google sign-in family, several document routes and the webhook path among them. The ones that are covered were chosen

because they carry money, authorisation or session state, which is the right priority, but "all endpoints are tested" would be false and is not claimed here.

6.15.5 Defects Found

Driving every endpoint against a real database, and the application against a real browser, turned up sixteen defects the unit tests had missed. Five were found in the first round of verification and are listed below; eleven more came out of the work that followed and are listed after them.

Table 6.2 — Defects Found by the End-to-End Runs

Defect	Severity	Status
Two of three endpoints accepting a room identifier checked ownership; the third did not, allowing one landlord's tenant to be filed into another landlord's room	Cross-account write	Fixed; the check is now one shared helper
A wrong password rendered "Your session expired", overriding the server's own message	User-facing	Fixed
Five required-field omissions produced HTTP 500 with a raw database error instead of a 400	Robustness	Fixed
Renaming a property without its address crashed on a NOT NULL column coerced on insert but not on update	Robustness	Fixed
A network failure reaching the gateway was reported as a rejected credential	User-facing	Fixed; transport failures now distinguished

The first one is worth dwelling on, since it is why ownership checks were centralised. Every read in the system is scoped by owner, so the second landlord never saw the intruding row. The leak was that the first landlord's own tenant list then displayed a stranger's property name and room number. We found it by filing one landlord's tenant into another's room through the API and getting a cheerful success response back.

Two more defects turned up in the test harnesses themselves, which is worth putting on the record. The first browser run never got past the permissions prompt, and because its assertions were phrased as the absence of a string that the prompt also happened to lack, half the suite passed while testing nothing at all. Separately, the failure reporter crashed on a failure whose context was undefined and took down the run it was meant to be reporting. The lessons are to assert on what should be there rather than on what should not, and to keep the failure reporter the dullest code in the file.

The second round of verification, run as each later module was built, found eleven more.

Table 6.3 — Defects Found During Later Verification

Defect	Severity	Status
Registering as a landlord returned no session token, so a new landlord was sent to the sign-in screen to type the password they had just chosen	User-facing	Fixed; registration now issues the same session a sign-in does
The landlord registration endpoint enforced no minimum password length, while the tenant endpoint required six characters	Security	Fixed server-side, where it cannot be skipped
Every signup failure was reported in one banner naming all four rules, leaving the reader to find which field was wrong	User-facing	Fixed; each field states its own fault and the banner counts them
The screen title sat twenty-two pixels off centre, and at the top of every tenant screen the bar held one button with the rest of its width empty	User-facing	Fixed; the suite now measures position, not just text
Both portals shared one create-account screen, so a landlord was asked whether they were over eighteen and could be shown a guardian-consent card	User-facing	Fixed; each role has its own form
Answering "under eighteen" promised a guardian would be texted a consent link; nothing was collected and no such message existed anywhere in the system	Correctness	Fixed; the guardian's details are collected and stored, and the wording says what happens
The guided tour started from only one of the four ways into a portal, so a registering user met it on their next launch instead	Correctness	Fixed
The tour's close control and the notifications bell were icon-only and had no accessible name, announcing as "button" and nothing else	Accessibility	Fixed; both name themselves, and the bell's name carries its state
The bell's new name called a non-empty list "something new", which would have reported news every day of the week about a date that had not moved	Correctness	Fixed before release; caught by the test written for it
A full verification run signed in often enough to trip the authentication rate limiter, after which every request returned an error indistinguishable in a log from a broken endpoint	Harness	Fixed; the limits are configurable, with the production values as defaults and a test proving they still bite
The focus clamp measured the header band from the top of the window, so with a notice bar above the header a field being typed into could scroll underneath it	Accessibility	Fixed; anything above the header now reports its height to the layout rule

A third round found four more, and found them in an unusual way: by checking this report and the repository's README against the code, rather than by running anything. Each document states what the system does; agents were set to verify every such

statement against the source. Four of the statements were false, and in each case it was the software that was wrong rather than the sentence.

Table 6.4 — Defects Found by Verifying the Documentation Against the Code

Defect	Severity	Status
The guided tour still did not start after registering — the one path of the four that the earlier fix was recorded as having covered	Correctness	Fixed; registration now starts it like the other three
A tenant under eighteen had their guardian's name and number stored and read back by nothing: no query anywhere selected those columns, so the landlord never saw them and the form's promise was still untrue	Correctness	Fixed; the guardian now appears on the join request, where the landlord decides
The update prompt had stopped listing what was in an update, because a refactor changed the phase it is shown in and the condition gating the notes was never updated with it	User-facing	Fixed
One suite loaded a dependency by an absolute path into one machine's scratch directory, so it could run there and nowhere else	Harness	Fixed; resolved relative to the repository

The first of those is the one worth dwelling on. Table 6.3 records the tour's entry points as fixed, and the commit message said four paths were covered. Three were. Nothing failed, no test went red, and the claim sat in this report as evidence of thoroughness until something checked it. A defect recorded as fixed is worse than a defect recorded as open, because it stops anybody looking.

The exercise generalises past this project. Documentation is normally reviewed for clarity, and its factual claims are taken on trust because checking them is tedious. Checking them mechanically turned out to be a verification technique in its own right, and a cheap one: it found four defects that every test suite in the build had passed over, including one in a feature that had been written, tested and released three days earlier.

The last one is the most instructive defect in the project, and it is worth the paragraph. The clamp exists for one purpose: to stop a field somebody is typing in from scrolling under the header. It worked out the forbidden band as the safe-area inset plus the header's height — correct for as long as the header was the first thing below the system bars. The new-build notice sits above the header, so while it is showing, that band ends well above where the header actually begins, and the clamp allowed too much scroll.

What makes it worth recording is not the bug but how it surfaced. The test suite that should have caught it was making the identical assumption: it measured "the header" as

the top fifty-eight pixels of the window. Both were wrong in the same direction, for the same reason, and neither could detect the other. It took a third thing — a bar that had never existed before — to put them out of step. A test that shares its subject's assumptions does not verify them, and nothing in a green run says which assumptions those are.

6.16 Testing Strategy

The numbers are in section 6.15. This section is about the choices behind them, because several were unusual and two were mistakes.

6.16.1 Why there is no test framework

Every suite is a plain Node script with a function that prints a line and increments a counter. There is no runner, no assertion library and no configuration file.

The reason is the environment rather than principle. A suite with no dependencies runs anywhere Node runs, needs no install step before it can be used, and cannot break because a runner changed its reporting format between versions. The cost is real and should be stated: there is no parallel execution, no filtering to a single test by name, and no machine-readable output, so wiring these into a continuous integration system will mean adding at least the last of those.

One claim made about this arrangement turned out to be untrue for one suite, and the fact-checking described in section 6.15.5 is what found it: a suite had been left loading a dependency by an absolute path into one machine's scratch directory, so "runs anywhere Node runs" was false of it. It now resolves the dependency relative to the repository.

6.16.2 Seeded data, and designing data that breaks things

The end-to-end suites run against a database built by a generator with a fixed seed, so a failing run can be reproduced exactly rather than approximately.

What the generator produces matters more than that it is deterministic. It deliberately creates the rows that break screens and that never appear when you are looking for them: a tenant with no room, a guest with no email address, a guest whose stay has already ended, rent that is overdue and rent due today and rent never set at all, a payment declared and undecided, another rejected with a reason, maintenance threads where the landlord spoke last and others where the tenant did, a tenant who has given notice, and a property with no rooms in it.

This is the opposite of the usual instinct, which is to seed data that looks tidy. Tidy data tests the happy path twice and the interesting paths never.

6.16.3 Driving the compiled application

The browser suites run the same JavaScript the phone runs. An export for web produces the identical view-model, the identical screens and the identical request client, laid out by a renderer and driven through a browser engine.

The limits are worth stating plainly, because a green run is persuasive. Nothing behind a native module is exercised: not the camera, not the image picker, not push delivery, and not the hand-off to a UPI application. Those are the parts of the system a tenant touches with money in play, and no assertion in this build covers any of them. Running a desktop browser is not a substitute for a device and is not claimed as one; it is the only automated way available in this project's environment to assert that a real screen renders a real row correctly, and it found defects that no unit test had.

6.16.4 Verifying across two sessions at once

One suite is unlike the others. The cross-portal suite opens two browser contexts, signs a tenant into one and a landlord into the other, and then asserts something neither side can assert alone: that a maintenance request raised on one appears on the other, and how long it takes.

Two numbers come out of it, and both are reported because they answer different questions. A ticket reaches the landlord in about two seconds if the landlord touches the screen, because any navigation refetches. It reaches an untouched screen in about nine, which is wherever in the application's polling cycle the request happened to land. Quoting only the first would overstate the system; quoting only the second would understate it.

This suite also produced a lesson about measurement. An early version navigated the landlord's screen while waiting, and so measured the first number while appearing to measure the second. A timing measurement that starts on an unknown screen measures nothing, and the suite now checks it is where it thinks it is before the clock starts.

6.16.5 Checking the documentation against the code

The last technique was not planned as one. Both this report and the repository's README state what the system does, and those statements were checked against the source by agents instructed to verify rather than to proofread.

Thirty statements did not survive. Four of them were false because the software was wrong rather than the sentence, and are recorded in Table 6.4. The remainder were errors in the documents and have been corrected.

The result that justifies calling this a verification technique rather than an editing pass is the first row of that table. A defect had been recorded as fixed, in this report and in a commit message, and three of its four cases had been. Every suite passed. Nothing was red. The claim stood as evidence of thoroughness until something read it against the code, and a defect recorded as fixed is worse than one recorded as open, because it stops anybody looking.

CHAPTER 7 — RESULTS AND SCREENSHOTS

The finished system meets the requirements set out in Chapter 5. A landlord can run several properties and their rooms, track every tenant's dues, confirm or reject payments, handle maintenance threads and connect a payment gateway. A tenant can see what they owe, pay it by either route, follow their own history, raise maintenance requests and give notice. The screenshots in this chapter come from the running application working against a seeded database of three properties, six rooms and ten tenants. None of the figures in them are placeholder text.

7.1 Landlord Overview

The landing screen answers the two questions that make a landlord open the application at all: who has paid, and who has not. Collection is shown against what was expected rather than as a bare total, because a number with no denominator is not something anyone can act on. Overdue gets a card of its own with a count of people attached. The revenue series is built from confirmed payments only. Claims that have been declared but not yet confirmed are left out on purpose, since counting them would inflate income on nothing more than a tenant's word. The bell carries a live count of whatever is waiting on a decision.

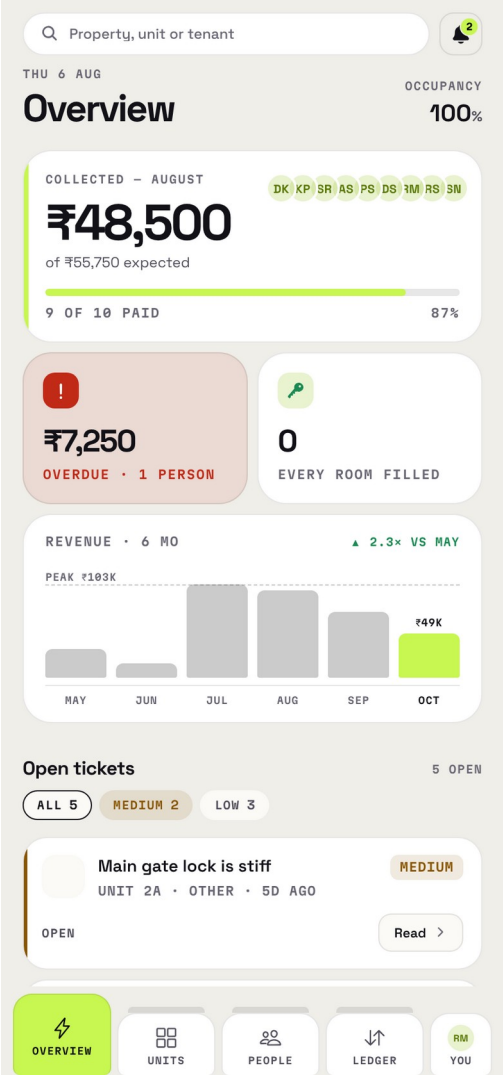


Fig 7.1 — Landlord overview: collection against expectation, overdue total, occupancy, six-month revenue and the open-ticket queue

7.2 Tenant Roster

Each row carries the tenant's property and room, their share of the rent, a payment-behaviour score and a due-state chip. The filter chips carry counts, so the spread is visible before any of them is tapped. The unassigned filter is not an afterthought. A tenant is allowed to exist without a room, and the end-to-end suite checks that such a row renders without a broken room label.

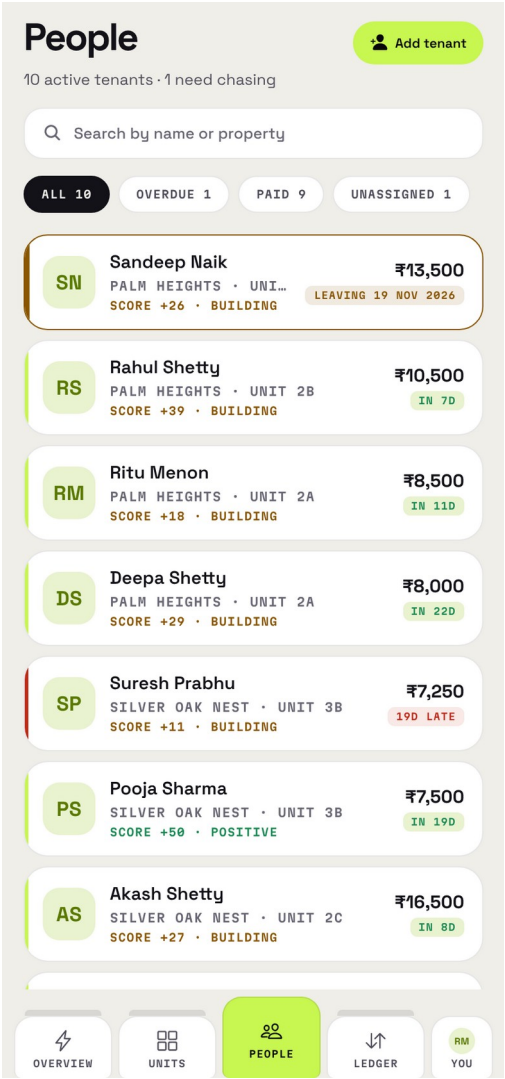


Fig 7.2 — Tenant roster with counted filters, due-state chips and behaviour scores

7.3 Rooms and Properties

Properties hold rooms and rooms hold tenants, each room with its own rent, capacity, and a split across occupants that can be equal or set by hand. A room's status follows its occupancy on its own. Notification preferences are per room, with each channel switched independently, because a landlord rarely wants the same channel across a whole portfolio.

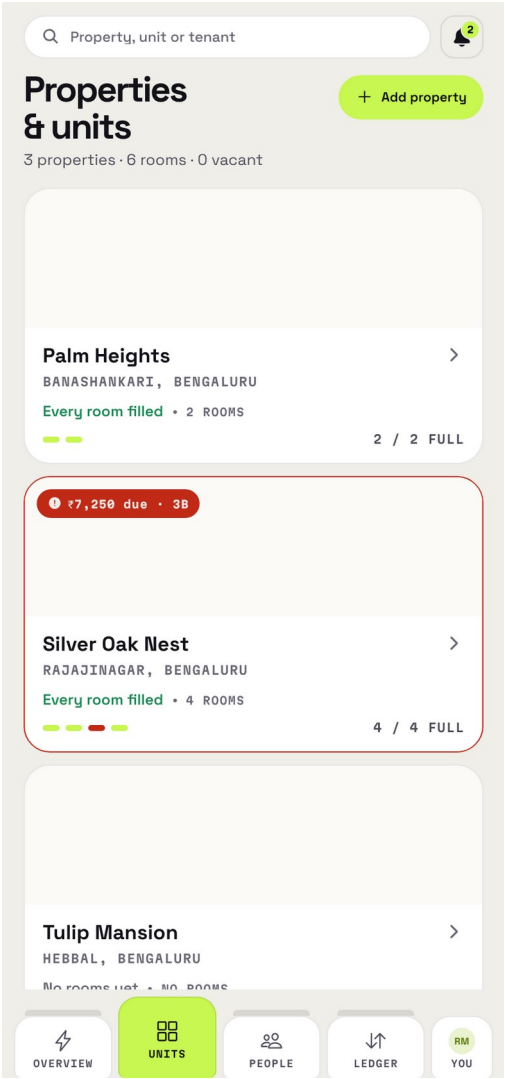


Fig 7.3 — Room list showing occupancy, rent and tenant assignment per unit

7.4 Ledger

The ledger holds every confirmed payment, searchable and narrowable to a single property. Each payment stores the due date it settled, captured at the moment of declaration, and that is the field that makes punctuality answerable after the fact.

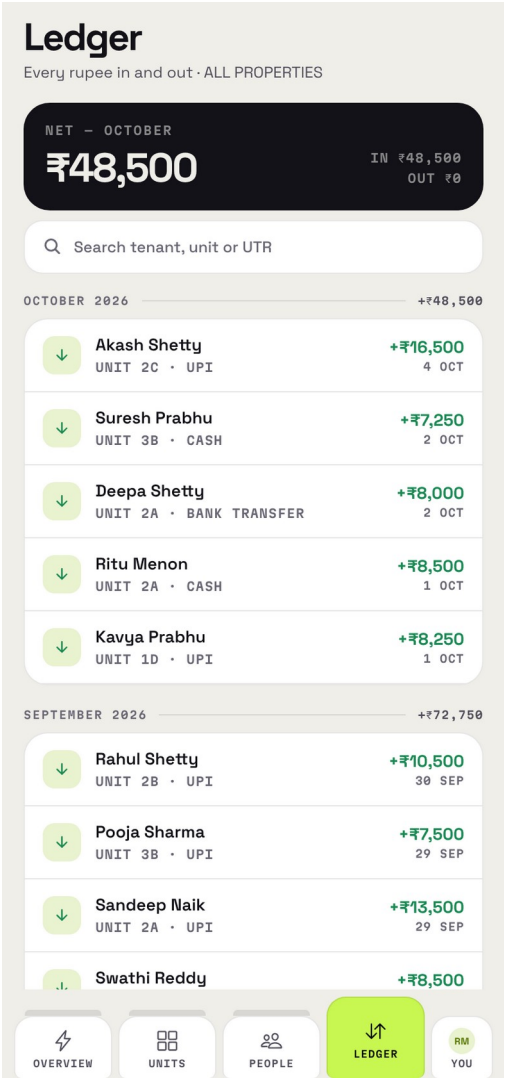


Fig 7.4 — Ledger of recorded payments with method, date and tenant

7.5 Connecting a Payment Gateway

The sheet states the fee next to the offer, and sets out what the gateway will ask for and how long it will take before the landlord begins. Credentials are checked against the gateway before being stored and sealed afterwards.

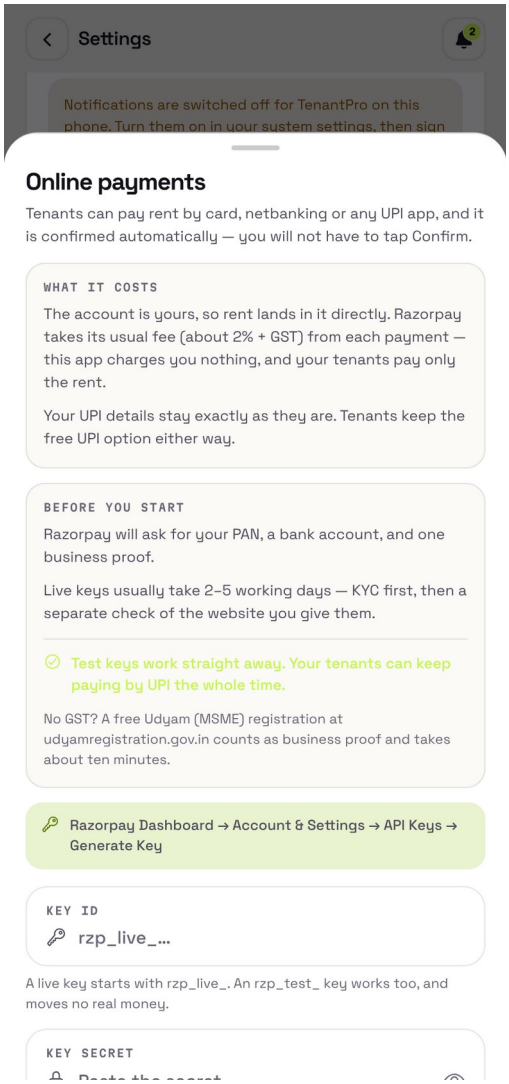


Fig 7.5 — The online payments sheet: what it costs, what the gateway will ask for, and the key fields

7.6 Tenant Home

The rent card carries the amount, the property and room, the exact due date and a countdown to it. Deposit held and tenure sit underneath, with the landlord's contact and any open requests after them. The alert indicator works off the same payload the screen renders rather than a separate counter.

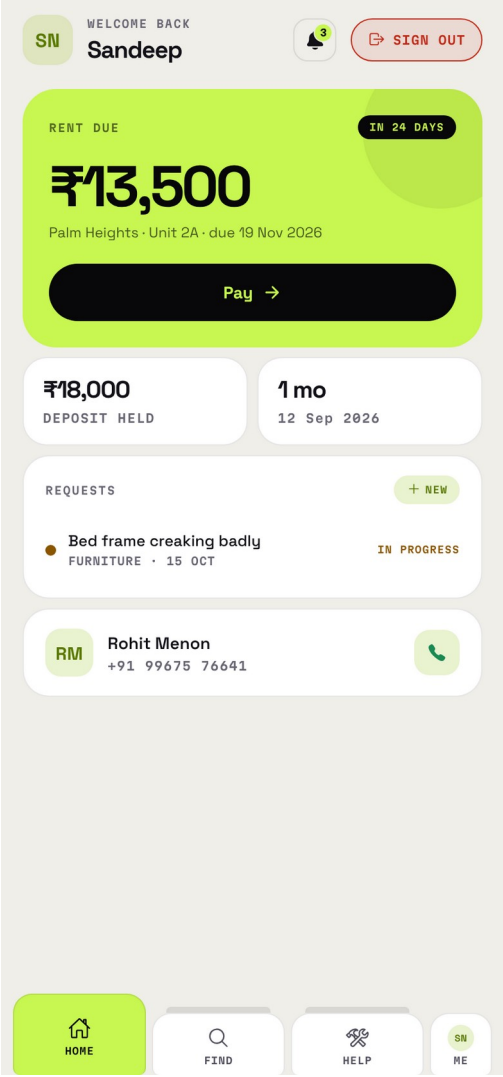


Fig 7.6 — Tenant home: rent due with countdown, deposit held, tenure, open requests and the landlord's contact

7.7 Paying Rent

The screen shows what is owed, a reference, and zero as the platform fee, which is literally true on both routes. Where the landlord has connected a gateway, the card and netbanking route comes first, since it is the only one that confirms itself. Below a divider sits the UPI route, with the landlord's own address, the amount and the reference encoded for scanning from another phone and as a hand-off to a UPI application on this one. The two buttons are styled differently on purpose: two filled primary buttons amount to two primary actions, which is the same as having none. Coming back from a gateway page, the button reads "Check my payment" rather than "Yes, I paid", because on that route the answer is not the tenant's to give.

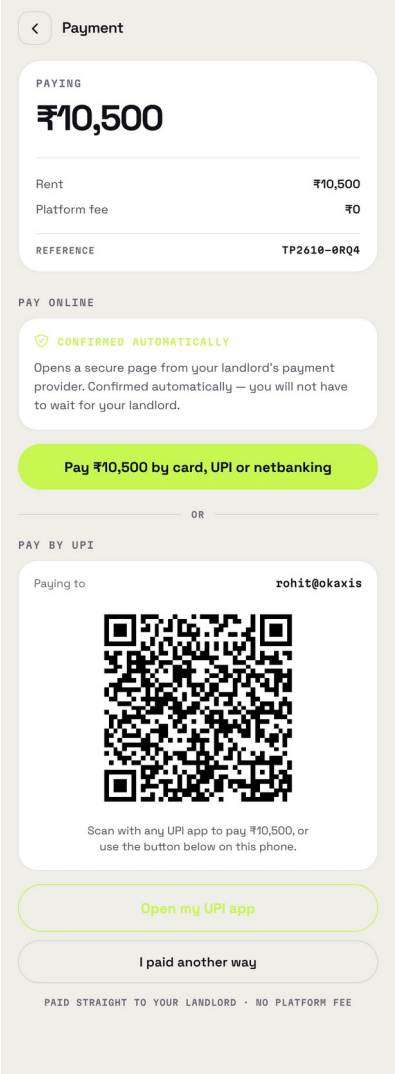


Fig 7.7 — The payment screen with a gateway connected: the online route above the divider and the UPI route beneath it

7.8 Tenant Profile and Payment History

The profile pulls together tenure, rent share, deposit and the behaviour score with its factors listed out, followed by the payment timeline. The timeline is built from real rows, and a claim still waiting on the landlord shows amber rather than the green kept for money that has arrived.

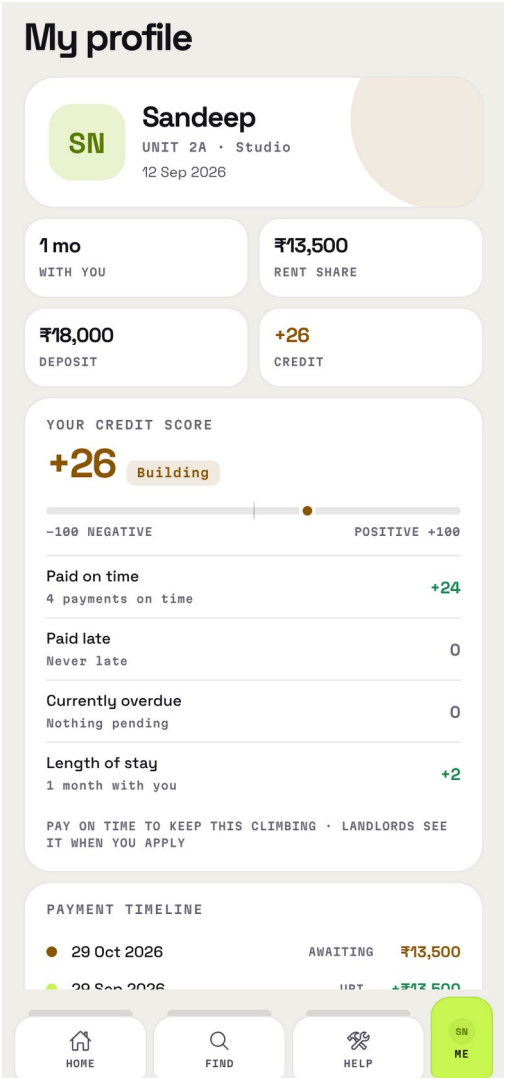


Fig 7.8 — Tenant profile: tenure, rent share, deposit, the behaviour score with its factors, and the payment timeline

7.9 Maintenance

A tenant raises a request with a category, a priority and an optional photograph, and follows it in a thread shared with the landlord, with status changes appearing in sequence alongside the conversation.

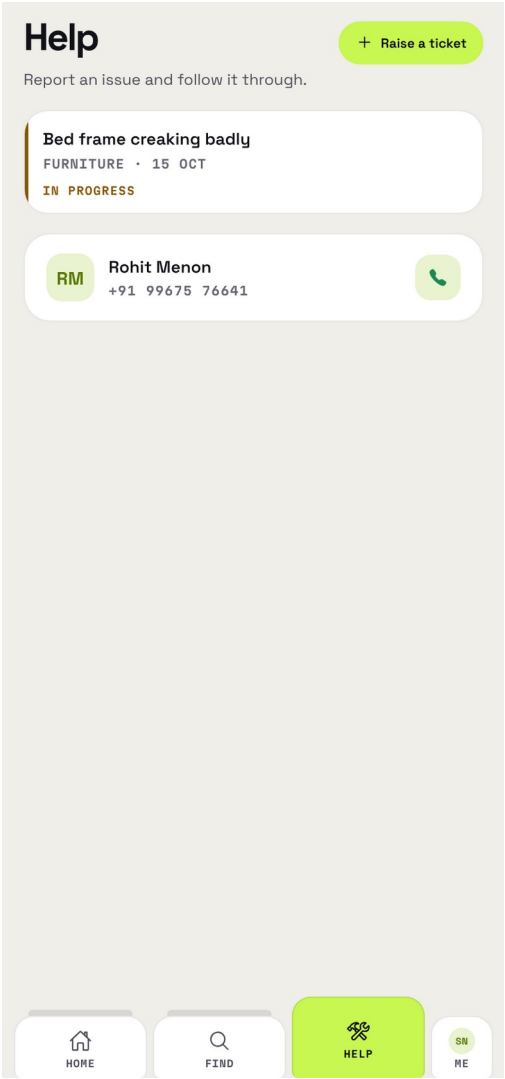


Fig 7.9 — Tenant help screen listing maintenance requests with their status

CHAPTER 8 — CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

This project set out to deal with a specific and long-standing gap in the independent rental sector: manual record keeping still dominates, and because of it there is no shared, authoritative account of what a tenant owes and what they have paid. What we built to address it is a working two-sided application, deployed and in use, made of a React Native client and a Node and Express API over MySQL, running to 55,499 lines across 178 source files, 96 endpoints, 23 tables and 28 screens.

What separates it from the alternatives reviewed in Chapter 2 is that the platform never touches the money. Rent goes straight from the tenant to the landlord, by UPI or through the landlord's own merchant account, and the platform only states the correct amount and destination and records what happened. That keeps the operator outside the payment-aggregator perimeter, costs both parties nothing, and leaves settlement where the landlords we spoke to wanted it. The price is that the system cannot observe money it does not handle, and the confirmation model exists to pay that price honestly rather than to pretend it does not apply.

The verification work produced the most useful findings of the project. Exercising much of the API against a real database and the compiled application against a real browser engine turned up sixteen defects that unit tests over mocks had not, among them a cross-account authorisation hole that let one landlord's tenant be written into another landlord's room, a signup form that asked a landlord their age, and a screen that promised to send a message the system had no means of sending. Two more were in the test harnesses rather than the application, and later rounds turned up a third.

The pattern across all of them is worth stating, because it is the project's clearest finding. Every one of these got past a green test run, and in more than one case the test and the code it was checking held the same wrong belief — most plainly in the focus clamp, where both the application and its suite measured the header from the top of the window, and only the arrival of a bar above the header put the two out of step. Tests that assert on the absence of a string, systems only ever tested against their own mocks, and tests that

share their subject's assumptions all give the same false confidence, and a passing run never says which of the three it is giving.

More broadly, the project shows that an independent landlord does not have to choose between a paper register and handing custody of their rent to a platform. A system can give both sides a complete, shared record while leaving the money exactly where it already flows.

8.2 Limitations of the Present Work

These are set out plainly, since each one follows from a design decision rather than from an oversight. The application cannot observe money. It never sees a bank balance, so a UPI payment stays a claim until the landlord confirms it, and for a landlord who has not connected a gateway the confirmation step is structural rather than incidental. Revocation lags by up to two hours on the access path, because closing that gap completely would mean a database lookup on every authenticated request.

Reuse detection can produce a false positive. A client that stores a new access token but fails to store the refresh token issued with it will present a spent token and lose every session. The client-side write is transactional, but a device that loses its storage halfway through would still land here. SMS is unconfigured, because phone verification needs a telecom registration that has not been done; email and push are both live. No iOS build has ever been produced, the bundle identifier is unset, and the project has only ever turned out Android artifacts.

Gateway onboarding is heavy. A landlord connecting Razorpay has to get through that gateway's own verification, realistically two to five working days, and although the application now says so before they start, it is still the biggest piece of friction in the product. Finally there is no automated regression gate. The suites exist and they pass, but nothing runs them on push, so verification stays a manual step somebody has to decide to take.

Three limitations were added by the later work and should be recorded alongside the rest. Push notifications have never been confirmed arriving on a real handset. The delivery path is built and the server reports no device has ever registered, which is consistent with it never having been exercised rather than with it being broken, but the honest position is that the in-application bell is verified and the notification tray is not. Everything the

browser-driven suites assert is asserted about the same JavaScript a phone runs, so the gap is narrower than it would otherwise be, but a browser is not a handset and nothing here claims otherwise.

A second payment provider was investigated and not built. The work stalled on two things outside the project's control: the provider's documentation could not be reached from the environment the work was done in, and a sandbox merchant account requires credentials that only the operator can obtain. What exists is a written account of what connecting one would involve, which is a specification rather than an implementation, and this report does not claim otherwise.

Third, the user-facing wording has been reduced but not finished. An audit of every string on every screen proposed seventy-one changes and confirmed fifty-two of them; twenty-eight were applied. The ones held back are as interesting as the ones taken, and are discussed under future work.

8.3 Future Enhancement

Several directions have been identified. Some are small and specified; others are larger and need more thought.

The immediate items are well understood. Reply notifications on maintenance threads are incomplete: a tenant replying to a repair request currently notifies nobody, though the landlord-to-tenant direction already works. The landlord's own payment-entry path inserts the payment and updates the due date in two separate statements where the tenant-facing path is already transactional, and that asymmetry should go. The suites are reproducible and fast enough to run on every push, so wiring them into continuous integration is configuration work rather than development work. An iOS build needs a bundle identifier and a first run through the pipeline. And a landlord announcement feature, a one-to-many message to everyone in a property, was specified during the notification work and then never built, although the alert infrastructure that would carry it already exists.

The larger directions need more care. Split settlement through a gateway's linked-account facility would let landlords onboard from inside the application, with the gateway doing their verification, which would remove the onboarding friction named above. The cost is that funds would then pass through a platform account, and that is a change of regulatory category rather than of configuration. The 2025 directions restrict such arrangements in

ways that call for professional advice rather than inference. Worth revisiting when the user base justifies it. Not before.

Monetisation, if it is ever pursued, should stay out of the money path. The Namma Yatri precedent applies here almost directly: a flat per-landlord subscription keeps the operator outside the payment flow, whereas a percentage of each transaction starts to look structurally like aggregation. There is a related legal nuance, which is that models of this kind rest on selling access to software rather than on setting the price of the underlying service. In TenantPro the landlord sets the rent, and that is a position worth holding on to deliberately.

Two smaller pieces of work are already identified on the messaging side. Transactional mail currently goes out from a consumer address through a provider's HTTPS interface; a verified sending domain with the usual authentication records would improve inbox placement a good deal, for about a thousand rupees a year. And phone verification needs an Indian telecom registration before any operator will carry template traffic.

The later rounds of work added several more directions, and a few lessons that should shape them.

Confirming delivery on a handset is the first and the least glamorous. The notification path is written and nothing has ever registered a device against it, so the whole chain from a rent reminder to a tray on somebody's phone is unproven. That needs an installed build on a real handset, a registered token and one reminder observed arriving, and it should be done before any further work on notifications rather than after.

The second payment provider should be finished once a sandbox account exists. The written account of what it involves is complete enough to work from, and the connection layer the system already has was built to take more than one provider, so this is implementation against a known design rather than a new design.

Verification on a device deserves the same treatment as verification in a browser. The browser-driven suites cover a great deal and were written knowing exactly where they stop: anything behind a native module — the camera, the image picker, the payment hand-off to a UPI application, the notification tray — is skipped rather than claimed. A device-driven run would close that, and would be the only way to assert that the parts of this system a tenant actually touches on the street behave as the browser says they do.

The wording audit should be finished, and finished carefully, because of what the first pass found. Agents reading every screen proposed seventy-one cuts and three independent checks confirmed fifty-two; of those, fifteen were rejected on a closer read. The rejections fell into a pattern nobody predicted. The checks were willing to cut the sentence that tells somebody how to recover from an error — removing "pull down to retry" from a failed load — and the reassurance attached to a destructive control, removing "no other landlord is touched" from a reset confirmation. One proposed deleting a field's label outright. The lesson generalises past this application: shorter and better diverge exactly where the reader is about to act, on an error or on something irreversible, and brevity measured without reference to what the reader is about to do will reliably cut the wrong sentence.

Accessibility should be audited as its own pass rather than caught in passing. Two of the controls in Table 6.3 had no accessible name, and both were found because a test could not reach them either — the test and the screen reader were blocked by the same omission. That coincidence is useful and should not be relied on. Every icon-only control in the application deserves checking directly, along with contrast, focus order and the behaviour of the alignment rule under a larger system font size, which has not been examined at all.

Two findings from the documentation check deserve their own line of work. The showcase listings that solve the discovery screen's cold start are not marked as samples anywhere a tenant can see, and they carry ordinary names and stock portraits, so they read as real. They should carry a visible marker, which is a small change and the clearest honesty issue remaining in the product. And the check itself should be repeatable: verifying a document's factual claims against the source found four defects that every suite had passed over, which makes it a verification technique rather than a proofreading exercise, and one worth running whenever either document is touched.

Two documentation items close the list. The report and the repository's own README have both drifted behind the code more than once during this project, which is an argument for generating the counts in them — lines, endpoints, tables, assertions — from the repository at build time rather than typing them and hoping. And the test plan published alongside this report predates nearly half the suites now in the build; it should be regenerated from the suites themselves for the same reason.

REFERENCES

[1] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

[3] Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.

[4] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

[5] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

[6] Nielsen, J. (1993). Usability Engineering. Morgan Kaufmann Publishers.

[7] Reserve Bank of India. Guidelines on Regulation of Payment Aggregators and Payment Gateways, and subsequent amendments. Retrieved from https://www.rbi.org.in/

[8] Commentary on the Regulation of Payment Aggregators Directions, 2025 — industry analysis of the revised framework for payment aggregators.

[9] Razorpay Developer Documentation. Payment Links API. Retrieved from https://razorpay.com/docs/payments/payment-links/

[10] Razorpay Developer Documentation. API Keys, and the website verification required for live mode. Retrieved from https://razorpay.com/docs/

[11] Razorpay. Route: split settlement and linked accounts. Retrieved from https://razorpay.com/docs/payments/route/

[12] Razorpay. KYC documents required by business type. Retrieved from https://razorpay.com/docs/

[13] National Payments Corporation of India. UPI Linking Specification and Deep Linking Guidelines. Retrieved from https://www.npci.org.in/

[14] Namma Yatri. Zero-commission direct-payment model — public documentation of the open mobility initiative. Retrieved from https://nammayatri.in/

[15] Internet Engineering Task Force. RFC 7519: JSON Web Token (JWT). Retrieved from https://www.rfc-editor.org/rfc/rfc7519

[16] Internet Engineering Task Force. OAuth 2.0 Security Best Current Practice, including refresh token rotation and replay detection. Retrieved from https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics

[17] National Institute of Standards and Technology. SP 800-38D: Recommendation for Block Cipher Modes of Operation — Galois/Counter Mode (GCM). Retrieved from https://csrc.nist.gov/

[18] Expo Documentation. Expo Updates and EAS Build. Retrieved from https://docs.expo.dev/

[19] React Native Documentation. Retrieved from https://reactnative.dev/

[20] Node.js Foundation. Node.js Official Documentation. Retrieved from https://nodejs.org/en/docs/

[21] Express.js. Express Web Framework Documentation. Retrieved from https://expressjs.com/

[22] Oracle. MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/

[23] Microsoft. Playwright Documentation — browser automation for end-to-end testing. Retrieved from https://playwright.dev/

Note on secondary sources. Several of the regulatory points in this report rest on consultancy and industry commentary rather than on the Reserve Bank of India's own text. Where the report leans on them, particularly for the net-worth thresholds and the restrictions on split settlement, the claims should be checked against the Directions themselves before anybody makes a structuring decision. The code figures, endpoint counts and assertion totals were measured directly from the repository and from live test runs, and are not estimates.

APPENDIX A — API ENDPOINT INVENTORY

A.1 How to read this appendix

Every route the server declares, grouped by the router that declares it and listed with the method, the full path as mounted, and the handler that answers it. It is generated from the source rather than maintained by hand, because a hand-kept list of ninety-six routes is a list that is wrong within a month. The count here is therefore the count in the code at the commit this report documents.

Two entries are worth noting before the tables. The route at the server root returns a plain string and exists so that a deployment platform's health probe has something to hit. And one development route sits behind a check on the environment, so a production deployment exposes ninety-four of the ninety-six listed.

A.2 Landlord authentication (/api/auth)

Table A.1 — Landlord authentication

Method	Path	Handler
POST	/api/auth/register	registerOwner
POST	/api/auth/login	loginOwner
POST	/api/auth/forgot-password	forgotPassword
POST	/api/auth/reset-password	resetPassword
POST	/api/auth/refresh	refreshToken
POST	/api/auth/logout	logout
POST	/api/auth/google/start	startGoogle
GET	/api/auth/google/callback	googleCallback
POST	/api/auth/google/poll	pollGoogle
POST	/api/auth/google/complete	completeGoogle

A.3 Landlord data (/api/owner)

Table A.2 — Landlord data

Method	Path	Handler
PUT	/api/owner/profile	updateProfile
PUT	/api/owner/profile/verify	verifyProfileContact
GET	/api/owner/pulse	getPulse
GET	/api/owner/dashboard	getDashboardStats
GET	/api/owner/transactions	getAllTransactions

Method	Path	Handler
GET	/api/owner/requests	getRequests
PUT	/api/owner/requests/:id/status	updateStatus
GET	/api/owner/requests/:id/messages	getMessages
POST	/api/owner/requests/:id/messages	createMessage
GET	/api/owner/join-requests	getJoinRequests
PUT	/api/owner/join-requests/:id	decideJoinRequest
GET	/api/owner/tenants/:id/documents	getTenantDocuments
GET	/api/owner/join-requests/:id/documents	getApplicantDocuments
PUT	/api/owner/documents/:id	decideDocument
POST	/api/owner/tenants/:id/id-request	createIdRequest
DELETE	/api/owner/tenants/:id/id-request	cancelIdRequest
POST	/api/owner/push-token	savePushToken
DELETE	/api/owner/push-token	dropPushToken
POST	/api/owner/push-test	sendTestPush
GET	/api/owner/demo	getDemoStatus
POST	/api/owner/demo/reset	resetDemo

A.4 Payments (/api/payments)

Table A.3 — Payments

Method	Path	Handler
GET	/api/payments/settings	getPaymentSettings
POST	/api/payments/settings	savePaymentSettings
POST	/api/payments/gateway	connectGateway
PUT	/api/payments/gateway	toggleGateway
DELETE	/api/payments/gateway	disconnectGateway
GET	/api/payments/declared	getDeclaredPayments
PUT	/api/payments/declared/:id	decidePayment
POST	/api/payments/:id/payments	recordPayment

A.5 Properties (/api/properties)

Table A.4 — Properties

Method	Path	Handler
GET	/api/properties	getProperties
POST	/api/properties	addProperty
PUT	/api/properties/:id	updateProperty
DELETE	/api/properties/:id	deleteProperty

A.6 Tenant authentication (/api/tenant-auth)

Table A.5 — Tenant authentication

Method	Path	Handler
POST	/api/tenant-auth/register	registerTenant
POST	/api/tenant-auth/login	loginTenant

A.7 Tenant portal (/api/tenant-portal)

Table A.6 — Tenant portal

Method	Path	Handler
GET	/api/tenant-portal/me	getMe
POST	/api/tenant-portal/alerts/seen	markAlertsSeen
POST	/api/tenant-portal/claim-account	claimAccount
POST	/api/tenant-portal/push-token	savePushToken
DELETE	/api/tenant-portal/push-token	dropPushToken
POST	/api/tenant-portal/push-test	sendTestPush
GET	/api/tenant-portal/discover	discover
GET	/api/tenant-portal/discover/:id	discoverOne
GET	/api/tenant-portal/pulse	getPulse
GET	/api/tenant-portal/payments	getPayments
POST	/api/tenant-portal/payments	declarePayment
POST	/api/tenant-portal/payments/online	startOnlinePayment
POST	/api/tenant-portal/payments/:id/check	checkOnlinePayment
GET	/api/tenant-portal/leave	getLeavePlan
POST	/api/tenant-portal/leave	giveNotice
DELETE	/api/tenant-portal/leave	withdrawNotice
GET	/api/tenant-portal/requests	getRequests
POST	/api/tenant-portal/requests	createRequest
GET	/api/tenant-portal/requests/:id/messages	getRequestMessages
POST	/api/tenant-portal/requests/:id/messages	createRequestMessage
GET	/api/tenant-portal/property-lookup	lookupProperty
POST	/api/tenant-portal/join-requests	createJoinRequest
GET	/api/tenant-portal/join-requests	getMyJoinRequests
GET	/api/tenant-portal/documents	getMyDocuments
POST	/api/tenant-portal/documents	addMyDocument
DELETE	/api/tenant-portal/documents/:id	deleteMyDocument

A.8 Tenants (/api/tenants)

Table A.7 — Tenants

Method	Path	Handler
POST	/api/tenants	addTenant
PUT	/api/tenants/assign	assignUnit
GET	/api/tenants/unit/:unit_id	getTenantsByUnit
GET	/api/tenants	getAllTenants
PUT	/api/tenants/:id	updateTenant
PUT	/api/tenants/:id/move-out	Delete
DELETE	/api/tenants/:id	Delete
PUT	/api/tenants/:id/financials	updateFinancials
PUT	/api/tenants/:id/stay	updateStay
GET	/api/tenants/unassigned	getUnassignedTenants
PUT	/api/tenants/:id/assign	assignTenantToRoom

A.9 Rooms (/api/units)

Table A.8 — Rooms

Method	Path	Handler
GET	/api/units/available	getAvailableUnits
GET	/api/units	getUnits
POST	/api/units	addUnit
PUT	/api/units/:id	updateUnit
DELETE	/api/units/:id	deleteUnit
PUT	/api/units/:id/settings	updateUnitSettings
GET	/api/units/:id/listing	getListing
PUT	/api/units/:id/listing	saveListing

A.10 Gateway webhooks (/api/webhooks)

Table A.9 — Gateway webhooks

Method	Path	Handler
POST	/api/webhooks/payments/:provider	handlePaymentWebhook

A.11 Declared on the application directly

Table A.10 — Routes outside the routers

Method	Path	Note
GET	/api/app/version	Whether a newer store build exists, compared server-side
GET	/api/id-preview/:id	Short-lived link to a blurred identity document
GET	/	Health probe for the hosting platform
GET	/healthz	Full status: database, mail, push, SMS, uploads, gateways, discovery, announced build
GET	/api/test-cron	Development only; absent in production

APPENDIX B — DATABASE SCHEMA

B.1 How to read this appendix

The tables the server creates on first boot, with their columns and types as declared. Generated from config/schema.sql, which is the file the server actually runs, so this is the schema rather than a drawing of it. Twenty-two tables are declared there. A twenty-third, oauth_sessions, is created by the migration code in config/initDb.js instead, because it arrived after that file had taken over responsibility for changes to an existing database; a deployment therefore holds twenty-three.

Columns added after a table first shipped are applied separately by the migration helpers at boot and may not appear in the CREATE statement below; where that matters the text says so. The entity relationships are in Chapter 4 and are not repeated here.

B.2 owners

Table B.1 — owners

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
name	varchar(100) NOT NULL
email	varchar(100) NOT NULL
phone	varchar(15) NOT NULL
password_hash	varchar(255) NOT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()
profile_pic	varchar(255) DEFAULT NULL

B.3 properties

Table B.2 — properties

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
owner_id	int(11) NOT NULL
name	varchar(150) NOT NULL
property_type	enum('PG','Apartment','Independent House','Hostel') NOT NULL
address	text NOT NULL
locality	varchar(100) DEFAULT NULL
city	varchar(100) DEFAULT 'Bengaluru'
pincode	varchar(10) DEFAULT NULL

Column	Type and constraints
latitude	decimal(10,7) DEFAULT NULL
longitude	decimal(10,7) DEFAULT NULL
upi_id	varchar(50) DEFAULT NULL
image_url	varchar(255) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.4 units

Table B.3 — units

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
property_id	int(11) NOT NULL
unit_number	varchar(50) NOT NULL
room_type	varchar(50) DEFAULT 'Standard'
capacity	int(11) DEFAULT 1
rent_split_type	enum('Equal','Custom') DEFAULT 'Equal'
base_rent	decimal(10,2) NOT NULL
image_url	varchar(255) DEFAULT NULL
status	enum('Vacant','Occupied','Maintenance') DEFAULT 'Vacant'
notify_email	tinyint(1) DEFAULT 1
notify_sms	tinyint(1) DEFAULT 1
notify_whatsapp	tinyint(1) DEFAULT 0

B.5 tenants

Table B.4 — tenants

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
owner_id	int(11) NOT NULL
unit_id	int(11) DEFAULT NULL
status	enum('Active','Inactive') DEFAULT 'Active'
name	varchar(100) NOT NULL
phone	varchar(15) NOT NULL
email	varchar(100) DEFAULT NULL
aadhar	varchar(20) DEFAULT NULL
company	varchar(150) DEFAULT NULL
emergency_phone	varchar(15) DEFAULT NULL
deposit	decimal(10,2) DEFAULT 0.00
rent_share	decimal(10,2) DEFAULT 0.00

Column	Type and constraints
image_url	varchar(255) DEFAULT NULL
id_proof_url	varchar(255) DEFAULT NULL
credit_score	int(11) DEFAULT 100
created_at	timestamp NOT NULL DEFAULT current_timestamp()
move_in_date	date DEFAULT NULL
billing_cycle	varchar(20) DEFAULT 'Anniversary'
next_rent_due	date DEFAULT NULL
stay_until	date DEFAULT NULL

B.6 document_requests

Table B.5 — document_requests

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
owner_id	int(11) NOT NULL
tenant_id	int(11) NOT NULL
doc_type	varchar(20) DEFAULT NULL
note	varchar(300) DEFAULT NULL
status	enum('Pending','Fulfilled','Cancelled') NOT NULL DEFAULT 'Pending'
created_at	timestamp NOT NULL DEFAULT current_timestamp()
closed_at	timestamp NULL DEFAULT NULL
document_id	int(11) DEFAULT NULL

B.7 push_tokens

Table B.6 — push_tokens

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
role	enum('owner','tenant') NOT NULL
account_id	int(11) NOT NULL
token	varchar(255) NOT NULL
platform	varchar(16) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()
last_seen_at	timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp()

B.8 leases

Table B.7 — leases

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
tenant_id	int(11) NOT NULL
unit_id	int(11) NOT NULL
start_date	date NOT NULL
end_date	date DEFAULT NULL
deposit_amount	decimal(10,2) NOT NULL
monthly_rent	decimal(10,2) NOT NULL
status	enum('Active','Notice Period','Moved Out') DEFAULT 'Active'

B.9 expenses

Table B.8 — expenses

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
property_id	int(11) NOT NULL
expense_category	varchar(50) NOT NULL
amount	decimal(10,2) NOT NULL
expense_date	date NOT NULL
description	text DEFAULT NULL

B.10 payment_settings

Table B.9 — payment_settings

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
owner_id	int(11) NOT NULL
upi_id	varchar(100) DEFAULT NULL
upi_number	varchar(15) DEFAULT NULL
qr_code_url	varchar(255) DEFAULT NULL
gateway	varchar(20) DEFAULT NULL
gateway_key_id	varchar(120) DEFAULT NULL
gateway_secret	varchar(512) DEFAULT NULL
gateway_enabled	tinyint(1) NOT NULL DEFAULT 0
updated_at	timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp()

B.11 rent_invoices

Table B.10 — rent_invoices

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
lease_id	int(11) NOT NULL
invoice_month	varchar(7) NOT NULL
base_amount	decimal(10,2) NOT NULL
late_fee	decimal(10,2) DEFAULT 0.00
total_due	decimal(10,2) NOT NULL
due_date	date NOT NULL
status	enum('Unpaid','Partial','Paid') DEFAULT 'Unpaid'

B.12 tenant_users

Table B.11 — tenant_users

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
name	varchar(100) NOT NULL
email	varchar(100) DEFAULT NULL
phone	varchar(15) NOT NULL
password_hash	varchar(255) DEFAULT NULL
tenant_id	int(11) DEFAULT NULL
status	enum('Unlinked','Pending','Linked') DEFAULT 'Unlinked'
is_guest	tinyint(1) NOT NULL DEFAULT 0
guest_code	varchar(16) DEFAULT NULL
alerts_seen_at	datetime DEFAULT NULL
guardian_name	varchar(120) DEFAULT NULL
guardian_relation	varchar(40) DEFAULT NULL
guardian_phone	varchar(20) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.13 payments

Table B.12 — payments

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
tenant_id	int(11) NOT NULL
amount_paid	decimal(10,2) NOT NULL
payment_date	date NOT NULL

Column	Type and constraints
due_date	date DEFAULT NULL
payment_method	varchar(50) NOT NULL
reference_id	varchar(100) DEFAULT NULL
status	enum('Declared','Confirmed','Rejected') NOT NULL DEFAULT 'Confirmed'
declared_by	int(11) DEFAULT NULL
decided_at	timestamp NULL DEFAULT NULL
decision_note	varchar(300) DEFAULT NULL
confirmation_source	enum('landlord','gateway','bank') NOT NULL DEFAULT 'landlord'
gateway_ref	varchar(120) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.14 password_resets

Table B.13 — password_resets

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
email	varchar(100) NOT NULL
role	varchar(10) NOT NULL DEFAULT 'owner'
code	varchar(255) NOT NULL
attempts	int(11) NOT NULL DEFAULT 0
expires_at	datetime NOT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.15 settlements

Table B.14 — settlements

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
lease_id	int(11) NOT NULL
move_out_date	date NOT NULL
total_deposit	decimal(10,2) NOT NULL
damage_deductions	decimal(10,2) DEFAULT 0.00
pending_rent_deductions	decimal(10,2) DEFAULT 0.00
final_refund_amount	decimal(10,2) NOT NULL
status	enum('Pending','Settled') DEFAULT 'Pending'

B.16 maintenance_requests

Table B.15 — maintenance_requests

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
tenant_id	int(11) NOT NULL
owner_id	int(11) NOT NULL
category	varchar(50) NOT NULL DEFAULT 'General'
title	varchar(150) NOT NULL
description	text DEFAULT NULL
priority	enum('Low','Medium','High') DEFAULT 'Medium'
status	enum('Open','In Progress','Resolved','Closed') DEFAULT 'Open'
image_url	varchar(500) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()
updated_at	timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp()

B.17 maintenance_messages

Table B.16 — maintenance_messages

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
request_id	int(11) NOT NULL
sender_role	enum('tenant','owner','system') NOT NULL
kind	enum('message','status') NOT NULL DEFAULT 'message'
body	text NOT NULL
status_from	varchar(20) DEFAULT NULL
status_to	varchar(20) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.18 join_requests

Table B.17 — join_requests

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
tenant_user_id	int(11) NOT NULL
owner_id	int(11) NOT NULL
property_id	int(11) NOT NULL
unit_id	int(11) DEFAULT NULL
status	enum('Pending','Accepted','Rejected') NOT NULL DEFAULT 'Pending'

Column	Type and constraints
note	varchar(300) DEFAULT NULL
requested_stay_until	date DEFAULT NULL
requested_unit_id	int(11) DEFAULT NULL
source	enum('code','phone_match') NOT NULL DEFAULT 'code'
created_at	timestamp NOT NULL DEFAULT current_timestamp()
decided_at	timestamp NULL DEFAULT NULL

B.19 tenant_documents

Table B.18 — tenant_documents

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
tenant_user_id	int(11) NOT NULL
doc_type	varchar(20) NOT NULL
doc_number	varchar(64) DEFAULT NULL
file_url	varchar(500) NOT NULL
status	enum('Pending','Verified','Rejected') NOT NULL DEFAULT 'Pending'
verified_by	int(11) DEFAULT NULL
verified_at	timestamp NULL DEFAULT NULL
note	varchar(300) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.20 demo_state

Table B.19 — demo_state

Column	Type and constraints
id	tinyint(4) NOT NULL DEFAULT 1
owner_id	int(11) DEFAULT NULL
last_reset_at	timestamp NULL DEFAULT NULL
reset_count	int(11) NOT NULL DEFAULT 0

B.21 contact_verifications

Table B.20 — contact_verifications

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
role	enum('owner','tenant') NOT NULL
account_id	int(11) NOT NULL

Column	Type and constraints
channel	enum('phone','email') NOT NULL
value	varchar(150) NOT NULL
code_hash	varchar(255) NOT NULL
expires_at	datetime NOT NULL
attempts	tinyint(4) NOT NULL DEFAULT 0
consumed_at	datetime DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()

B.22 sessions

Table B.21 — sessions

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
token_hash	char(64) NOT NULL
account_kind	enum('owner','tenant') NOT NULL
account_id	int(11) NOT NULL
expires_at	datetime NOT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()
last_used_at	datetime DEFAULT NULL
replaced_at	datetime DEFAULT NULL
revoked_at	datetime DEFAULT NULL
revoked_reason	varchar(20) DEFAULT NULL
device	varchar(200) DEFAULT NULL

B.23 listings

Table B.22 — listings

Column	Type and constraints
id	int(11) NOT NULL AUTO_INCREMENT
unit_id	int(11) NOT NULL
owner_id	int(11) NOT NULL
published	tinyint(1) NOT NULL DEFAULT 0
headline	varchar(150) DEFAULT NULL
description	text DEFAULT NULL
available_from	date DEFAULT NULL
deposit_months	decimal(4,1) NOT NULL DEFAULT 1.0
preferred	enum('Anyone','Students','Working professionals','Families') NOT NULL DEFAULT 'Anyone'
furnishing	enum('Unfurnished','Semi-furnished','Fully furnished') NOT NULL DEFAULT 'Semi-furnished'

Column	Type and constraints
amenities	varchar(400) DEFAULT NULL
created_at	timestamp NOT NULL DEFAULT current_timestamp()
updated_at	timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp()

APPENDIX C — TEST SUITE INVENTORY

C.1 How to read this appendix

Every suite in the build, what it needs running, and what it asserts. Assertion counts are those a passing run prints, not counts of lines in the source, so a suite that skips an assertion on a platform that cannot reach it reports the smaller number.

None of these uses a test framework. Each is a plain Node script that exits non-zero when an assertion fails, which is why the whole set can be run from a shell loop and why a failing run is reproducible from one command.

Table C.1 — Every suite in the build

Suite	Needs	What it covers
api.e2e.js	Live server and MySQL	Not a unit test: nothing is stubbed. It signs in, reads what it was given, and
appversion.test.js	Nothing	Does the app know a newer BUILD exists?
gateway.test.js	Nothing	`fetch` is the interesting stub. Every request this code makes to Razorpay is captured
guardian.e2e.js	Live server and MySQL	A tenant under 18, end to end — form to column.
layout.test.js	Nothing	Three things in that file are pure functions of numbers, and all three are
ratelimit.test.js	Nothing	The limiter still limits.
refresh.client.test.js	Nothing	These assertions are all about not logging somebody out. The server half is tested
refresh.e2e.js	Live server and MySQL	The whole point of this feature is that a session can be ENDED. So most of these
secretbox.test.js	Nothing	—
session.test.js	Nothing	This is the bug the user actually hit — "Your session expired. Please sign in
ui.alerts.e2e.js	Live server, MySQL and a served web build	The bell: in-app notifications on both sides.
ui.crossportal.e2e.js	Live server, MySQL and a served web build	Does a tenant raising a problem actually reach their landlord — and how fast?
ui.discover.e2e.js	Live server, MySQL and a served web build	Drives the compiled app in Chromium: the place-discovery feature, the three
ui.e2e.js	Live server, MySQL and a served web build	Not the emulator the user asked for — this container has no KVM, so an Android

Suite	Needs	What it covers
ui.layout.e2e.js	Live server, MySQL and a served web build	The header/content alignment rule, in a real browser: does the screen's title
ui.signout.e2e.js	Live server, MySQL and a served web build	Does the app tell you it is signing out?
ui.signup.e2e.js	Live server, MySQL and a served web build	Signing up: the happy path, and every way of getting it wrong.
ui.signuproles.e2e.js	Live server, MySQL and a served web build	A landlord and a tenant are not signing up for the same thing.
ui.storebanner.e2e.js	Live server, MySQL and a served web build	"A new version is available" — on every screen, in both portals.
ui.tour.e2e.js	Live server, MySQL and a served web build	The guided tour: does it appear for somebody new, does every control do what
ui.updates.e2e.js	Live server, MySQL and a served web build	"Check for updates", on both portals.
vm.gateway.test.js	Nothing	deriveVm is pure — state and an api object in, a plain tree out — so this needs no